Practică SGBD

Ce învăţăm la acest curs ?

- Fiind vorba de practică… nimic (prea) teoretic.
- La curs sunt acoperite topicile:
  - cum optimizăm o interogare (indexare);
  - tranzacţii (o bucată ramasă de la BD);

- La laborator: învăţăm PL/SQL (Procedural Language / Structured Query Language)

“Viața” unei interogări

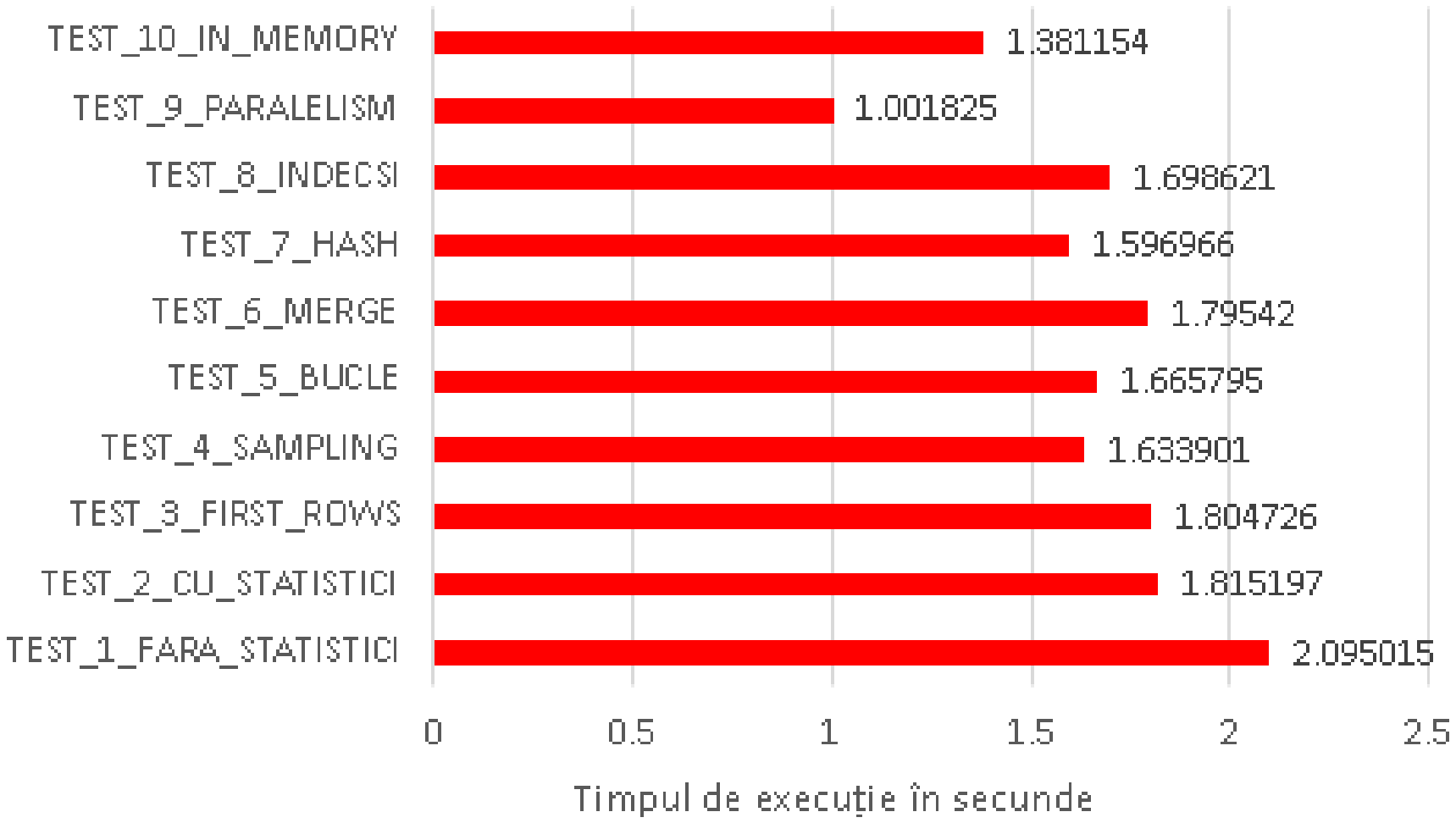
- Aplicația-client formulează interogarea
- Prin intermediul unui driver trimite interogarea către SGBD (indiferent care e el)
- Interogarea e parsată de SGBD la nivel sintactic și tranformată (e.g. “*” devine …)
- Sunt executate eventualele view-uri pentru a ști de unde să preia informația.
- Se creează un **plan de execuție** (sau este selectat unul anterior creat).

Cel mai important pas ?

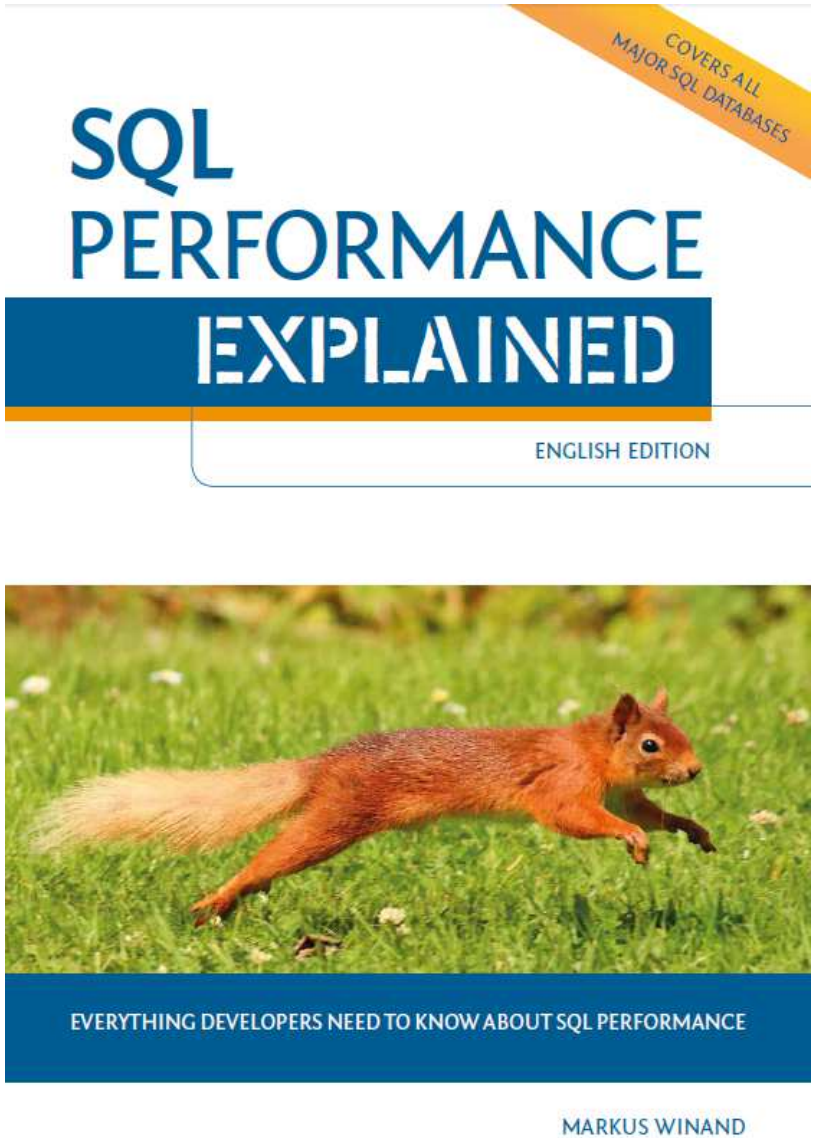
“Viața” unei interogări

- Sunt preluate datele de pe HDD si construit un răspuns.
- Răspunsul este returnat către client.

- În continuare vom vedea câteva statistici realizate pe un join între două tabele (1M randuri + 1K randuri).


TEST_10_IN_MEMORY
1.381154
TEST_9_PARALELISM
1.001825
TEST_8_INDECSI
1.698621
TEST_7_HASH
1.596966
TEST_6_MERGE
1.79542
TEST_5_BUCLE
1.665795
TEST_4_SAMPLING
1.633901
TEST_3_FIRST_ROWS
1.804726
TEST_2_CU_STATISTICI
1.815197
TEST_1_FARA_STATISTICI
2.095015
0
0.5
1
1.5
2
2.5
Timpul de execuţie în secunde

http://use-the-index-luke.com/



Sintactic & Semantic

- Putem considera o interogare SQL ca fiind o propoziţie din engleză ce ne indică ce trebuie făcut fără a ne spune cum este făcut:

```
SELECT prenume
   FROM studenti
   WHERE nume = 'POPESCU'
```

… şi dacă am un timp de răspuns de 15 secunde ?

La baza unei aplicaţii ce nu merge (sau merge greu) stau două greşeli umane*

- Autorului unei interogări SQL nu îi pasă (de obicei) ce se întâmplă “în spate”.
- Autorul interogării nu se consideră vinovat dacă timpul de răspuns al SGBD-ului este mare (evident, cel care l-a inventat nu prea a ştiut ce face).
- Soluţia ? Simplu: nu mai folosim Oracle, trecem pe MySQL, PostgreSQL sau SQL Server (că ne-a zis nouă cineva că merge mai bine).

***Una dintre ele este de a da vina pe calculator.**

De fapt…

- Singurul lucru pe care dezvoltatorii trebuie să îl înveţe este cum să indexeze corect (bine, poate nu singurul…).
- Cea mai importantă informaţie este felul în care aplicaţia va utiliza datele.
- Traseul datelor nu este cunoscut nici de client, nici de administratorul bazei de date şi nici de consultanţii externi; singurul care ştie acest lucru este dezvoltatorul aplicaţiei !

…cuprins… (legat de indexare)

- Anatomia unui index
- Clauza WHERE
- Performanţă şi Scalabilitate
- JOIN
- Clustering
- Sortare & grupare
- Rezultate parţiale
- INSERT, UPDATE, DELETE

Anatomia unui index

- *"An index makes the query fast"* - cât de rapid?

```
-----------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |     1 |     9 |     5   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| STUDENTI |     1 |     9 |     5   (0)| 00:00:01 |
-----------------------------------------------------------------------------


PLAN_TABLE_OUTPUT
-------------------------

   1 - filter("NUME"='Popescu')
```

Anatomia unui index

- *"An index makes the query fast"* (5x ?)

```
-----------------------------------------------------------------------------
| Id  | Operation        | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT |          |     1 |     9 |     1   (0)| 00:00:01 |
|*  1 |  INDEX RANGE SCAN| NUME_STD |     1 |     9 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------

PLAN_TABLE_OUTPUT
--------------------------------

   1 - access("NUME"='Popescu')
```

Anatomia unui index

- Un index este o structură* distinctă într-o bază de date ce poate fi construită utilizând comanda **`create index`**.

```
select index_name from user_indexes;
```

- Are nevoie de propriul spaţiu pe HDD şi *pointează* tot către informaţiile aflate în baza de date (la fel ca şi cuprinsul unei cărţi, redundantă până la un anumit nivel – sau chiar 100% redundant: SQL Server sau MySQL cu InnoDB folosesc *Index-Organized Tables* [IOT]).

* vom detalia pana la un anumit nivel (nu complet)

Anatomia unui index

- Căutarea după un index este asemănătoare cu căutarea într-o carte de telefon.
- Indexul din BD trebuie să fie mult mai optimizat din cauza dinamicităţii unei BD

  [insert / update / delete]
- Indexul trebuie menţinut fără a muta cantităţi mari de informaţie.

Timpul necesar căutării într-un fişier "sortat"

- Sa presupunem că avem 1.000.000 date de "dimensiune egală" [din acet motiv old SGBD gen FoxPro nu prea se impăca bine cu variabile ca cele de tip varchar2].
- Căutarea binară => $\log_2(1.000.000)$ =20 citiri
- Un HDD de 7200RPM face o rotaţie completă în 60/7200" = 0.008333.." = 8.33ms
- Pentru un Seagate ST3500320NS, track-to-track seek time = 0.8ms

https://en.wikipedia.org/wiki/B-tree

Timpul necesar căutării într-un fişier “sortat”

- Căutarea (de 0.8ms) şi citirea unei piste (8.33ms) să zicem că ajungem pentru o citire la 10ms.
- 20 citiri = 200ms = 0.2”
- Probabil timpul este mare pentru că ultimele informaţii se pot afla pe aceeaşi pistă ceea ce va “eficientizează” şi nu vom mai citi aceeaşi pistă ultimele 3-4 ture => 0.16”.

https://en.wikipedia.org/wiki/B-tree

Timpul necesar căutării într-un fişier “sortat”

- Dacă ar fi trebuit să caut 10 valori ? Ar fi fost necesare 2 secunde
- Dacă ar fi fost necesar să caut 100 de valori (care de obicei sunt afişate pe o pagina web gen e-bay).... 20 de secunde. Sigur nu v-ar plăcea să aşteptaţi un magazin online 20 de secunde până să vă afişeze cele 100 de produse ;) … şi OLX merge mai bine :D
- Nici nu vreau să mă gândesc ce s-ar întâmpla dacă datele nu ar fi sortate…

Cum obţin timpi şi mai mici ?

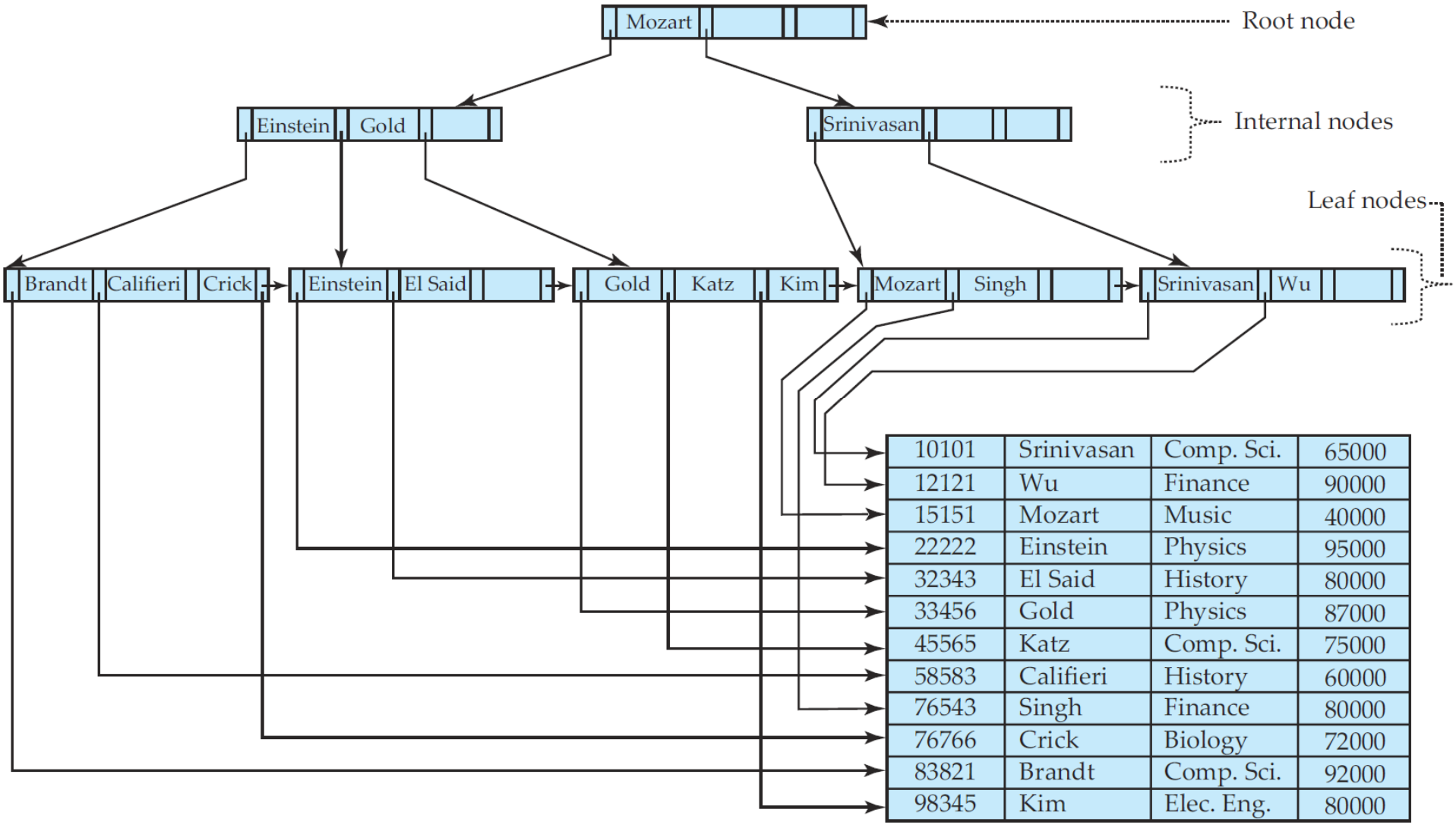
$\log_2(1.000.000)$

Pont: Încercaţi să schimbaţi asta…

Anatomia unui index

- Cum funcţionează ?
  - pe baza unui arbore de cautare
  - pe baza unei liste dublu înlănţuite
- Arborele este utilizat pentru a căuta datele indexate (B^+-trees)
- Prin intermediul listei se pot insera cantităţi mari de date fără a fi nevoie să le deranjăm pe cele existente.

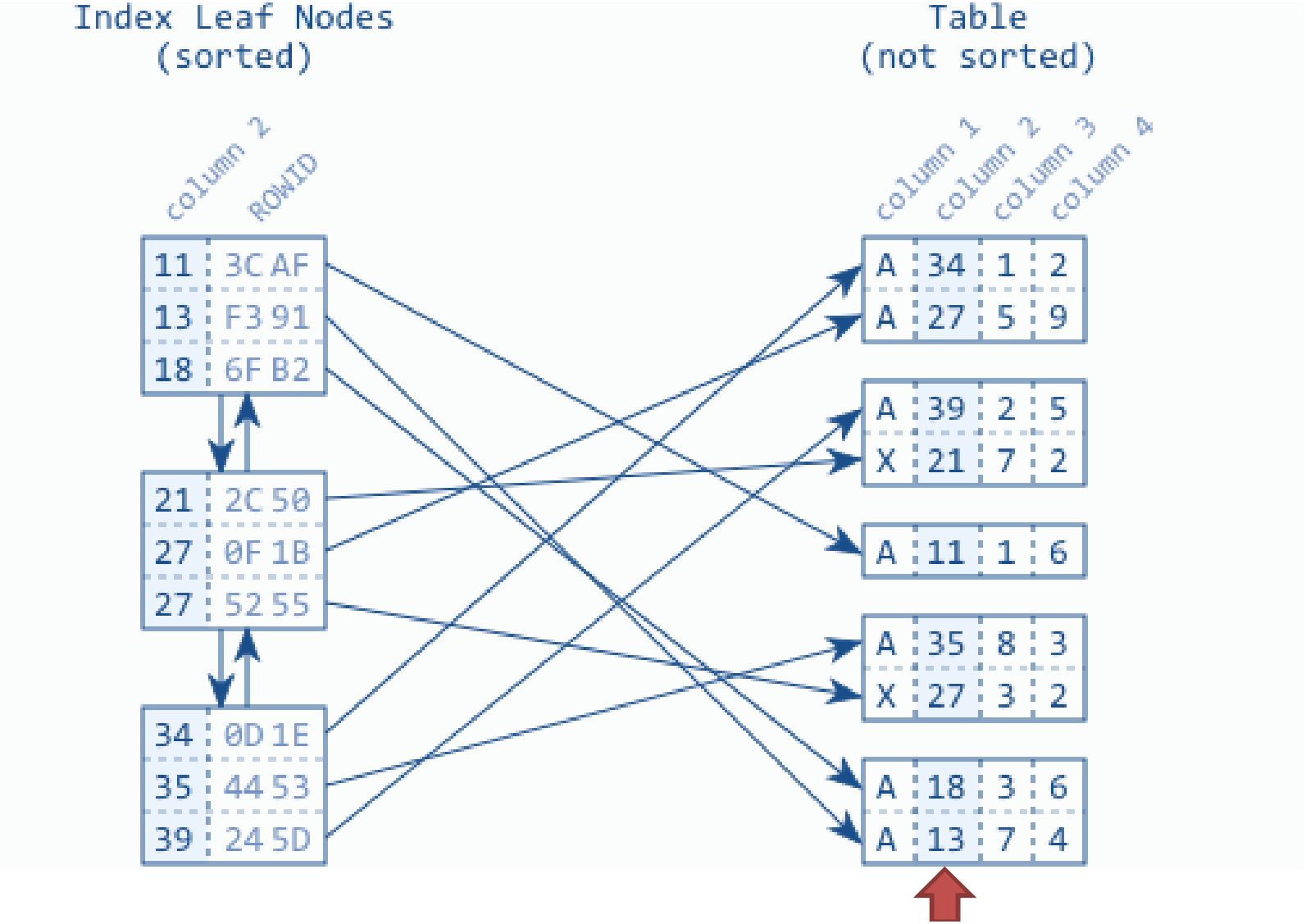
For a long time it was unclear what the "B" in the name represented. Candidates discussed in public where "Boeing", "Bayer", "Balanced", "Bushy" and others. In 2013, the B-Tree had just turned 40, Ed McCreight revealed in an interview, that they intentionally never published an answer to this question. They were thinking about many of these options themselves at the time and decided to just leave it an open question. http://sqlity.net/en/2445/b-plus-tree/


Mozart
Root node
Einstein
Gold
Srinivasan
Internal nodes
Leaf nodes
Brandt
Califieri
Crick
Einstein
El Said
Gold
Katz
Kim
Mozart
Singh
Srinivasan
Wu
10101 Srinivasan Comp. Sci. 65000
12121 Wu Finance 90000
15151 Mozart Music 40000
22222 Einstein Physics 95000
32343 El Said History 80000
33456 Gold Physics 87000
45565 Katz Comp. Sci. 75000
58583 Califieri History 60000
76543 Singh Finance 80000
76766 Crick Biology 72000
83821 Brandt Comp. Sci. 92000
98345 Kim Elec. Eng. 80000

Pentru o reprezentare mai facilă…

- Vom considera că numărul pointerilor dintr-un nod este egal cu cel al valorilor – fiecare pointer are valoarea cea mai mare din următorul nod (de fapt am ignorat unul din pointeri) !
- Frunzele conţin toate valorile şi nu trimit către un bucket ce conţine valorile de acelaşi fel (nu este neapărat adevărat, dar nici fals) !
- În practică lista de frunze poate fi dublu înlănţuită (pentru ca indexul să poată fi parcurs în sens invers).

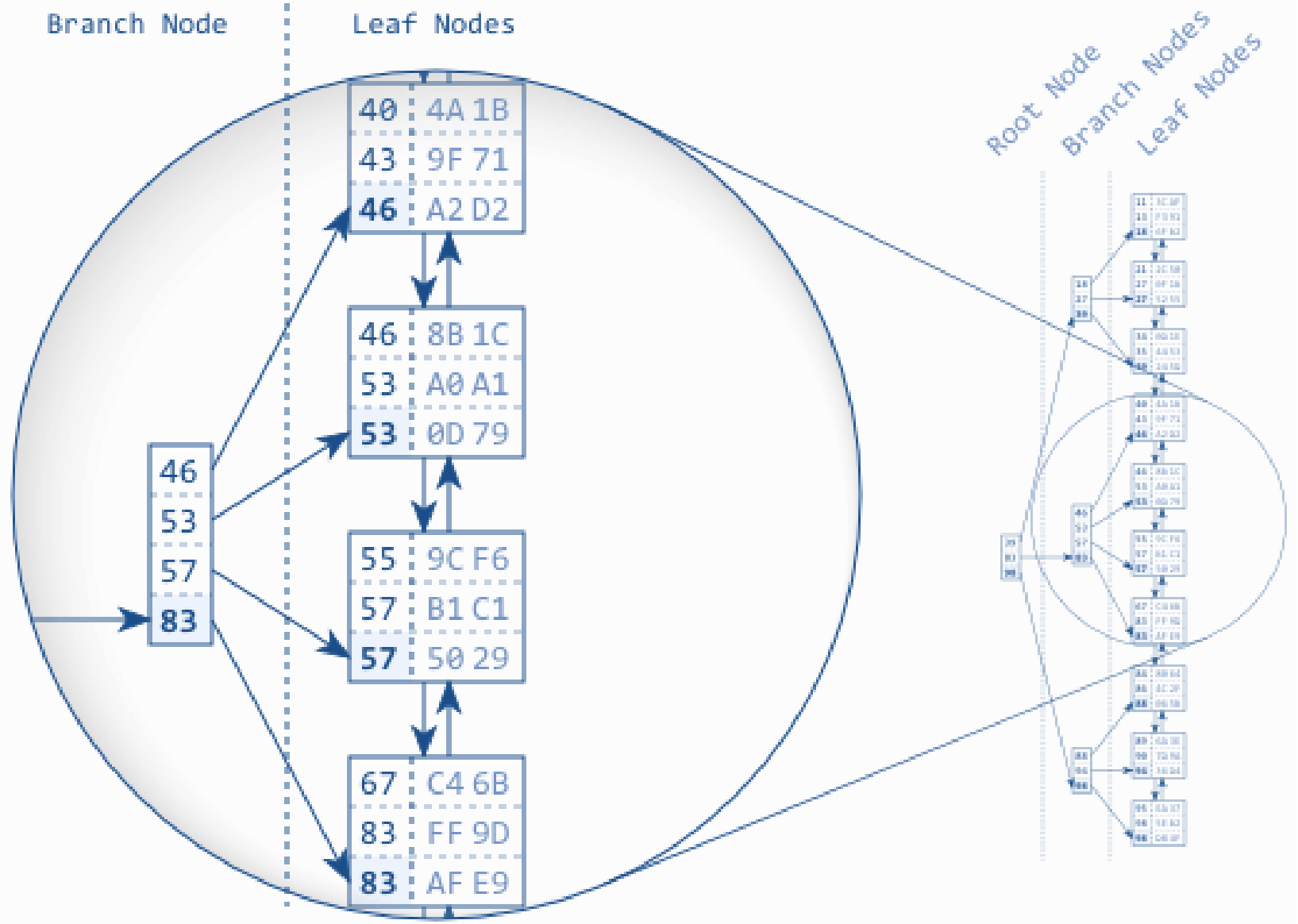
Anatomia unui index – frunzele pointează către informațiile din tabelă



Anatomia unui index

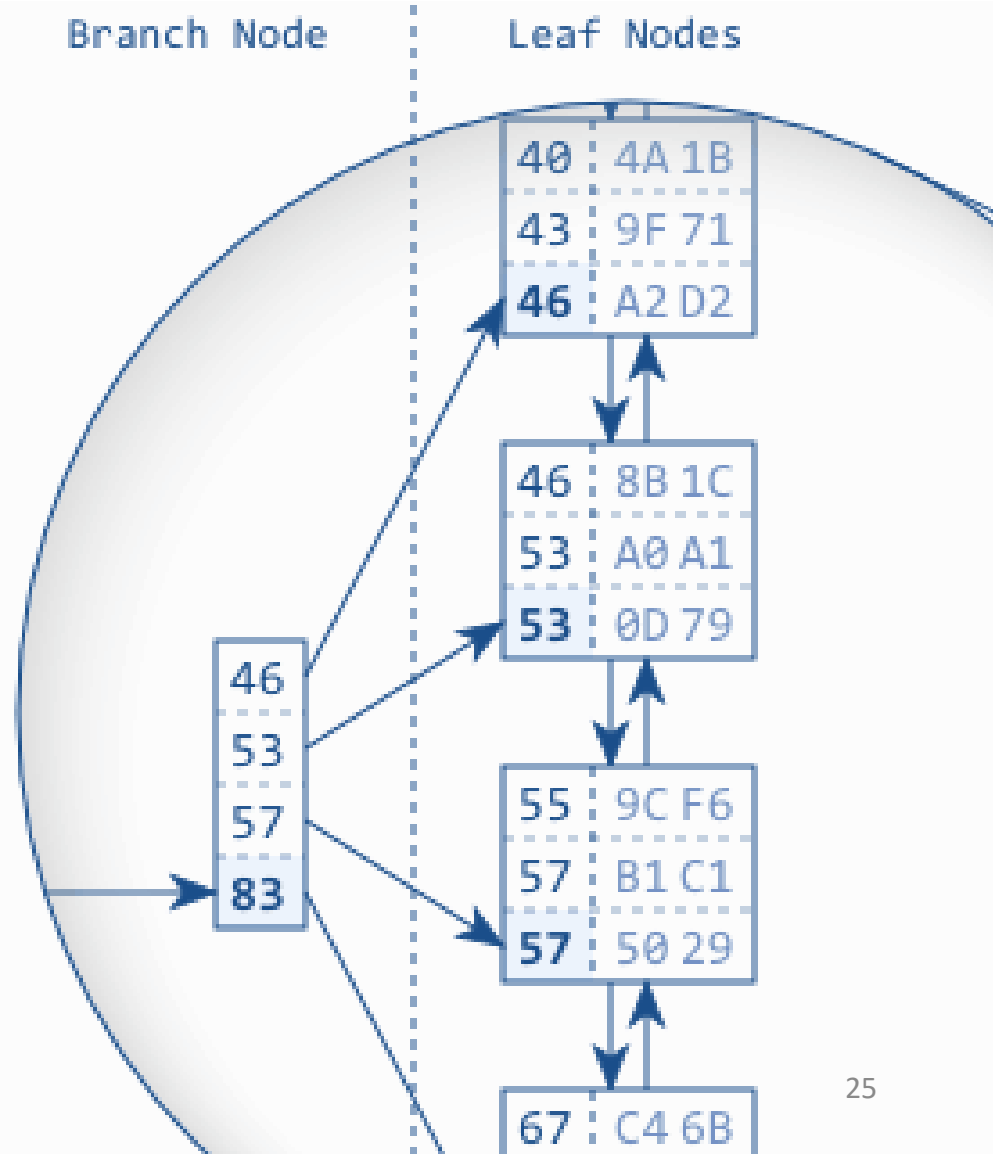
- “frunzele” nu sunt stocate pe disc în ordine sau având o aceeaşi distribuţie – poziţia pe disk nu corespunde cu ordinea logică a indecşilor (de exemplu, dacă indexăm mai multe numere între 1 şi 100 nu e neapărat ca 50 să se afle exact la mijloc) – putem avea 80% de valori 1.
- SGBD-ul are nevoie de cea de-a doua structură pentru a căuta rapid între indecşii amestecaţi.

Anatomia unui index

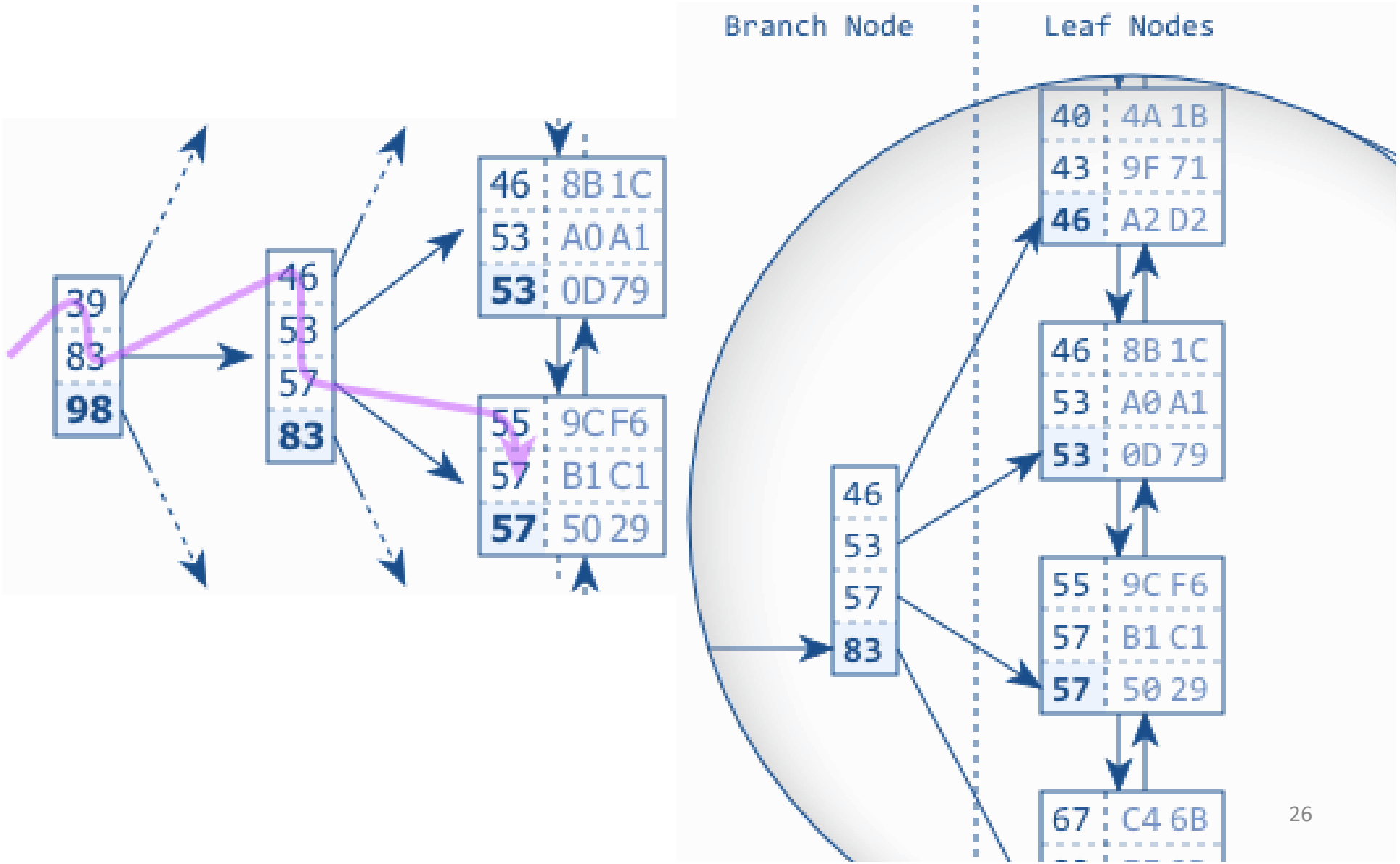


Anatomia unui index

Pointerul către următorul nivel indică cea mai mare valoare a acestui următor nivel.



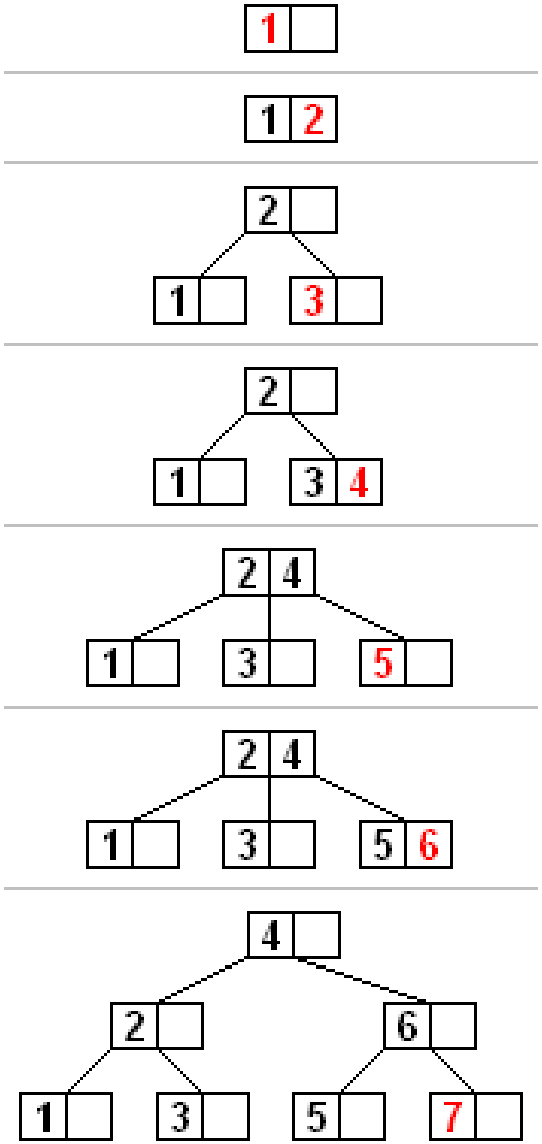
Anatomia unui index



Anatomia unui index

- Un *B+ tree* este un arbore echilibrat !
- Un *B+ tree* nu este un arbore binar !
- Adâncimea arborelui este identică spre oricare dintre frunze.
- Odată creat, baza de date menţine indecşii în mod automat, indiferent de operaţia efectuată asupra bazei de date (insert/delete/update)
- *B+ tree*-ul facilitează accesul la o frunză;
- Cât de repede ? [*first power of indexing*]

Cum se balansează un B+ tree ?



- Dacă ar avea maxim 2 locaţii pe nod ar putea să ajungă să se comporte ca un arbore binar…
- Având mai multe locaţii şi fiind “sparse”, e mult mai flexibil (observaţi că uneori rămâne echilibrat chiar după inserare)
- Dacă ar avea mai multe locaţii libere în fiecare nod, nevoia de echilibrare ar fi şi mai rară.

Cum se balansează un B+ tree ?

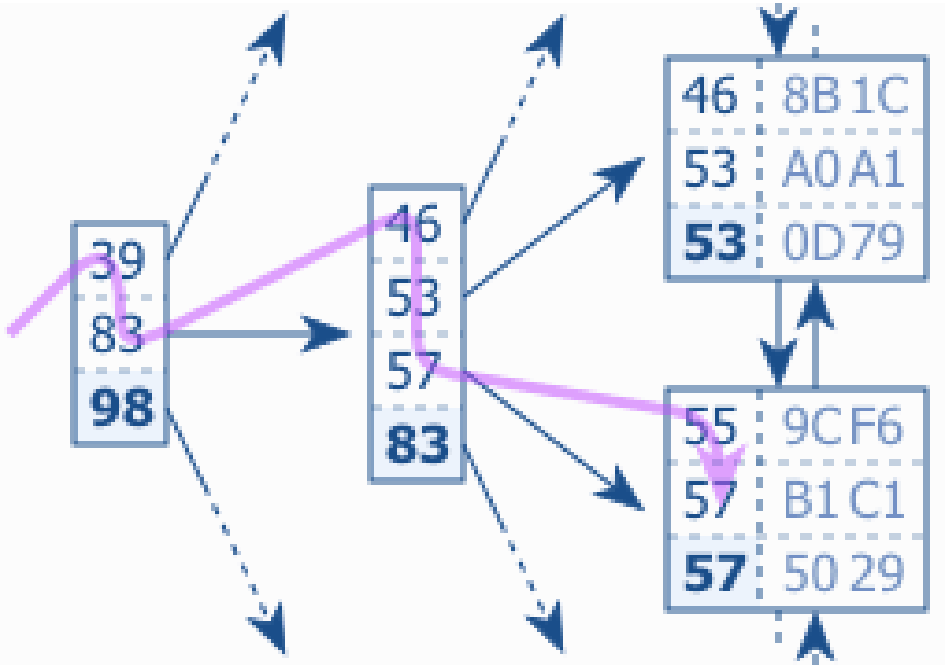
- Nu ne interesează la nivel formal (pentru asta aveţi cursuri de algoritmică/programare etc.)
- Ideea de bază este ca atunci când se ajunge la numărul maxim de valori într-un nod, el se scindează în două noduri şi se reface echilibrarea, când este eliminată ultima valoare, se reechilibrează în sens invers.
- Echilibrarea nu e neapărat să ajungă până în rădăcină ea putându-se face în valorile libere de până la rădăcină.

Anatomia unui index

- Deşi găsirea informaţiei se face în timp logaritmic, există “mitul” că un index poate degenera (şi ca soluţie este “reconstruirea indexului”). - fals deoarece arborele se autobalansează.

Anatomia unui index

- De ce ar funcţiona un index greu ?



- Atunci când sunt mai multe rânduri (57,57…) – (aici ar putea exista un “bucket”).

Anatomia unui index

- De ce ar funcţiona un index greu ?
- După găsirea indexului corespunzator, trebuie obţinut rândul din tabel.

- Căutarea unei înregistrări indexate se face în 3 paşi:
  - Traversarea arborelui [limita superioară: adâncimea arborelui: oarecum rapid]
  - Căutarea frunzei în lista dublu înlănţuită [încet]
  - Obţinerea informaţiei din tabel [încet]

Anatomia unui index

- Este o concepţie greşită să credem că arborele s-a dezechilibrat şi de asta căutarea este înceată. În fapt, traversarea arborelui pare să fie cea mai rapidă.
- Dezvoltatorul poate “întreba” baza de date despre felul în care îi este procesată interogarea.

Anatomia unui index

- În Oracle există trei tipuri de operaţii importante:

  `INDEX UNIQUE SCAN`

  `INDEX RANGE SCAN`

  `TABLE ACCESS BY INDEX ROWID`

- Cea mai costisitoare este INDEX RANGE SCAN.
- Dacă sunt mai multe rânduri, pentru fiecare dintre ele va face TABLE ACCESS – în cazul în care tabela este împrăştiată în diverse zone ale HDD, şi această operaţie devine greoaie.

Planul de executie

- Pentru a interoga felul în care Oracle procesează o interogare: **EXPLAIN PLAN FOR**

```
SQL> EXPLAIN PLAN FOR SELECT NUME FROM STUDENTI WHERE NUME='Popescu';

Explained.
```

- Pentru a afişa rezultatul, se execută

```
SELECT* FROM TABLE(dbms_xplan.display);
```

```
SQL> SELECT * FROM TABLE(dbms_xplan.display);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 953379482

-----------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |     1 |     9 |     5   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| STUDENTI |     1 |     9 |     5   (0)| 00:00:01 |
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------

   1 - filter("NUME"='Popescu')

Note
-----
   - dynamic sampling used for this statement (level=2)

17 rows selected.
```

```
SQL> SELECT * FROM TABLE(dbms_xplan.display);

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 1113182500

-----------------------------------------------------------------------------
| Id  | Operation        | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT |          |     1 |     9 |     1   (0)| 00:00:01 |
|*  1 |  INDEX RANGE SCAN| NUME_STD |     1 |     9 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------

   1 - access("NUME"='Popescu')

Note
-----
   - dynamic sampling used for this statement (level=2)

17 rows selected.
```

Oracle SQL Developer : C:\Users\Cosmin\AppData\Roaming\SQL Developer\Untitled8.sql

File Edit View Navigate Run Source Team Tools Window Help

Connections

- Connections
  - bd2_app
  - local
  - ServerTestPSGBD
  - startrek
  - student
- Oracle NoSQL Connections
- Cloud Connections

Start Page | Untitled7.sql | script_populare_student.sql | script_populare_student_alumni.sql | Untitled8.sql

SQL Worksheet | History

0 seconds

Worksheet | Query Builder

```
1  select * from studenti where nume='Popescu';
```

Query Result | Explain Plan

SQL | 0 seconds

OPERATION	OBJECT_NAME	CARDINALITY	COST
SELECT STATEMENT		1	4
TABLE ACCESS (BY INDEX ROWID)	STUDENTI	1	4
INDEX (RANGE SCAN)	NUME_STD	4	1

Access Predicates

NUME='Popescu'

Other XML

```
{info}
  info type="db_version"
    11.2.0.2
  info type="parse_schema"
  "STUDENT"
  info type="dynamic_sampling"
    2
  info type="plan_hash"
    2441470029
  info type="plan_hash_2"
    128465632
  {hint}
    INDEX_RS_ASC(@"SEL$1" "STUDENTI"@
    OUTLINE_LEAF(@"SEL$1")
    ALL_ROWS
    DB_VERSION('11.2.0.2')
    OPTIMIZER_FEATURES_ENABLE('11.2.0.
    IGNORE_OPTIM_EMBEDDED_HINTS
```

Clauza WHERE

Clauza **WHERE**

- Clauza **WHERE** dintr-un select defineşte condiţiile de căutare dintr-o interogare SQL şi poate fi considerată nucleul interogării – din acest motiv influenţează cel mai puternic rapiditatea cu care sunt obţinute datele.
- Chiar dacă **WHERE** este cel mai mare duşman (sau prieten) al vitezei, de multe ori este "aruncat" doar "pentru că putem".
- Un **WHERE** scris rău este principalul motiv al vitezei mici de răspuns a BD.

Clauza **WHERE**

```
CREATE TABLE studenti (
  id INT PRIMARY KEY,
  nume VARCHAR2(15) NOT NULL,
  prenume VARCHAR2(30) NOT NULL,
  data_nastere DATE,
  email VARCHAR2(40), … (LAB)
);
```

Index creat automat

…şi se adaugă 1025 de studenţi.

Clauza **WHERE**

```
SELECT nume, prenume
    FROM studenti
    WHERE id = 300
```

```
SQL> SELECT * FROM TABLE(dbms_xplan.display);
---------------------------------------------------------------------------------------
| Id  | Operation                   | Name         | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |              |     1 |    39 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI     |     1 |    39 |     2   (0)| 00:00:01 |
|*  2 |   INDEX UNIQUE SCAN         | SYS_C0014184 |     1 |       |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):

  2 - access("ID"=300)
```

E mai bine unique scan sau range scan ?

Un index creat pe un primary key poate avea range scan dacă este interogat cu egalitate ?

Clauza **WHERE**

```
SELECT nume, prenume
 FROM studenti
 WHERE id BETWEEN 200 AND 210
```

```
SQL> explain plan for select nume, prenume from studenti where id between 200 and 210;
SQL> SELECT * FROM TABLE(dbms_xplan.display);

---------------------------------------------------------------------------------------
| Id  | Operation                   | Name         | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |              |    11 |   429 |     4   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI     |    11 |   429 |     4   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | SYS_C0014184 |     5 |       |     2   (0)| 00:00:01 |
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access("ID">=200 AND "ID"<=210)
```

Concatenarea indecşilor

- Uneori este nevoie ca indexul să îl construim peste mai multe coloane:

```
CREATE UNIQUE INDEX idx_note ON
 note(id_student, id_curs);
```

- Căutarea va fi făcută după id_student. Informaţia din noduri/frunze va fi peste ambele câmpuri. Când se va ajunge la studentul cu un anumit id, cautarea în index va continua după id_curs:

```
SQL> EXPLAIN PLAN FOR SELECT * FROM note WHERE id_student=300 AND id_curs=1;

Explained.

SQL> SELECT * FROM TABLE(dbms_xplan.display);
---------------------------------------------------------------------------------------
| Id  | Operation                   | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |          |     1 |    79 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| NOTE     |     1 |    79 |     2   (0)| 00:00:01 |
|*  2 |   INDEX UNIQUE SCAN         | IDX_NOTE |     1 |       |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access("ID_STUDENT"=300 AND "ID_CURS"=1)

14 rows selected.
```

Clauza **`WHERE`**

- Atunci când cele două câmpuri ce intră în componenţa indexului formează o cheie candidat (unic / nenul), putem crea indexul cu CREATE **UNIQUE** INDEX ….
- Ce se întâmplă dacă vrem să căutăm doar după unul din câmpuri ?

caz 1: căutare după **`id_student`**

caz 2: căutare după **`id_curs`**

Căutare după câmpul **id_student**

```
SQL> explain plan for select * from note where id_student=300;

Explained.

SQL> SELECT * FROM TABLE(dbms_xplan.display);
---------------------------------------------------------------------------------
| Id  | Operation                   | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |          |     5 |   395 |     3   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| NOTE     |     5 |   395 |     3   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_NOTE |    65 |       |     2   (0)| 00:00:01 |
---------------------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access("ID_STUDENT"=300)
```

Căutare după câmpul **id_curs**

```
SQL> EXPLAIN PLAN FOR SELECT * FROM note WHERE id_curs=1;

Explained.

SQL> SELECT * FROM TABLE(dbms_xplan.display);
---------------------------------------------------------------------------
| Id  | Operation         | Name | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |      |  1007 | 79553 |    30   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| NOTE |  1007 | 79553 |    30   (0)| 00:00:01 |
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("ID_CURS"=1)
```

Clauza **WHERE**

- Dacă am considera că indexul nostru este peste o carte de telefon, atunci acesta ar indexa ca și prim câmp numele (de familie) şi apoi prenumele.
- Interogarea anterioara ar fi echivalentul căutării în cartea de telefon a tuturor abonaţilor cu prenumele “Vasile” – nu se poate face decât prin parcurgerea întregii cărţi de telefon.

Clauza **WHERE**

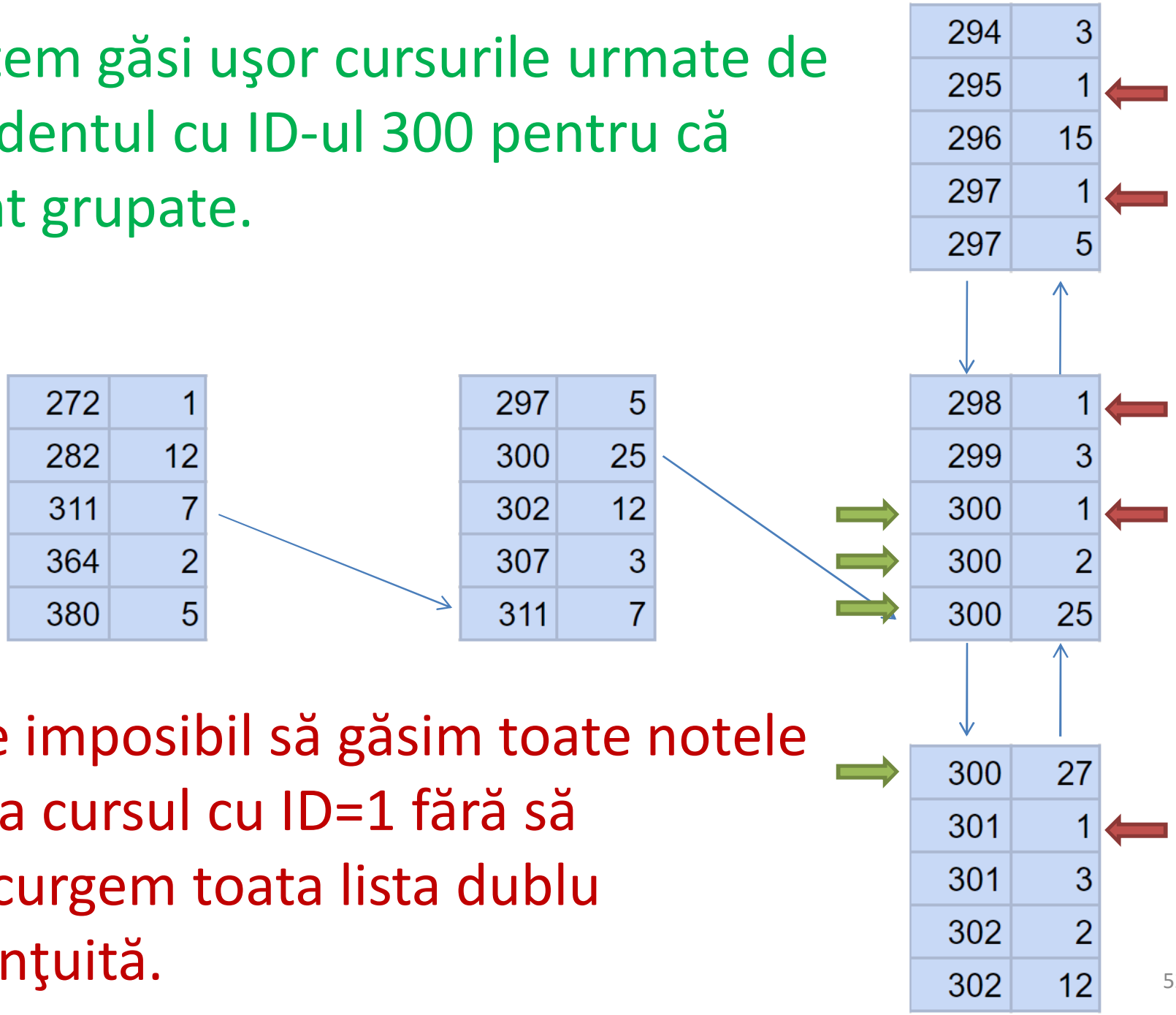
Indexul nu a fost utilizat.
Cu cât a crescut utilizarea procesorului ?

Operaţia este rapidă într-un exemplu mic, dar foarte costisitoare în caz contrar.

Clauza **WHERE**

- Uneori scanarea completă a bazei de date este mai eficientă decât accesul prin indecşi. Acest lucru este parţial chiar din cauza timpului necesar căutării indecşilor (e.g. parcurgerea listei înlănţuite nu ar fi mai rapidă decât parcurgerea tabelei note).
- O singură coloană dintr-un index concatenat nu poate fi folosită ca index (excepţie face prima coloană).

Putem găsi uşor cursurile urmate de studentul cu ID-ul 300 pentru că sunt grupate.



Este imposibil să găsim toate notele de la cursul cu ID=1 fără să parcurgem toata lista dublu înlănţuită.

Clauza **WHERE**

- Se observă că valoarea 1 pentru **id_curs** este distribuită aleator prin toată tabela. Din acest motiv, nu este eficient să căutăm folosind acest index.
- Cum facem ca să căutăm eficient?

```
DROP INDEX idx_note;
CREATE UNIQUE INDEX idx_note
 ON note(id_curs, id_student);
```

- În continuare indexul este format din aceleaşi două coloane (dar în altă ordine).

Clauza **WHERE**

- Cel mai important lucru când definim indecşi concatenaţi este să stabilim ordinea.
- Dacă vrem să utilizăm 3 câmpuri pentru concatenare, căutarea este eficientă (de fapt poate fi ajutată de index) pentru câmpul 1, pentru 1+2 şi pentru 1+2+3 dar nu şi pentru alte combinaţii.

Clauza **WHERE**

- Atunci când este posibil, este de preferat utilizarea unui singur index (din motive de spaţiu ocupat pe disc şi din motive de eficienţă a operaţiilor ce se efectuează asupra bazei de date).
- Pentru a face un index compus eficient trebuie ţinut cont şi care din câmpuri ar putea fi interogate independent – acest lucru este ştiut de obicei **doar de către programator**.

Indecşi “înceţi”

- Schimbarea indecşilor poate afecta întreaga bază de date ! (operaţiile pe aceasta pot deveni mai greoaie din cauză că managementul lor este făcut diferit)
- Indexul construit anterior este folosit pentru toate interogările în care este folosit **`id_curs`** şi pentru toate care folosesc id_curs, id_student (nu contează ordinea).

- Dacă avem doi indecşi disjuncţi şi în clauza WHERE sunt folosiţi ambii ? Pe care dintre ei îi va considera BD? Este mereu eficient să se ţină cont de indecşi ?

```
SQL> drop index idx_note;

Index dropped.

SQL> CREATE INDEX idx_note1 ON note(id_student);

Index created.

SQL> CREATE INDEX idx_note2 ON note(id_curs);

Index created.
```

```
SQL> EXPLAIN PLAN FOR SELECT * FROM note WHERE id_student=300 AND id_curs=1;

Explained.

SQL> SELECT * FROM TABLE(dbms_xplan.display);
---------------------------------------------------------------------------------------
| Id  | Operation                   | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT            |           |     2 |   158 |     2   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS BY INDEX ROWID| NOTE      |     2 |   158 |     2   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN          | IDX_NOTE1 |    65 |       |     1   (0)| 00:00:01 |
---------------------------------------------------------------------------------------

Predicate Information (identified by operation id):

   1 - filter("ID_CURS"=1)
   2 - access("ID_STUDENT"=300)

Note
-----
   - dynamic sampling used for this statement (level=2)

19 rows selected.
```

```
SQL> select count(*) from note
  2  where id_curs=1;

  COUNT(*)
----------
      1025

SQL> select count(*) from note
  2  where id_student=300;

  COUNT(*)
----------
        16
```

Indecşi “înceţi”

- ***The Query Optimizer***
- Componenta ce transformă interogarea într-un plan de execuţie (aka compiling / parsing).
- Două tipuri de opimizere:
  - *Cost based optimizers* (CBO) – mai multe planuri, calculează costul lor şi rulează pe cel mai bun;
  - *Rule-based optimizers* (RBO) – foloseşte un set de reguli hardcodat (de DBA).

CBO poate sta prea mult să caute prin indecşi şi RBO să fie mai eficient în acest caz [1000x1000 tbl]

Indecşi “înceţi”

- Să presupunem că am avea următorul scenariu (un simplu update în studenţi):

```
SQL> select count(grupa), grupa from studenti group by grupa;

COUNT(GRUPA) GR
------------ --
         995 Z
          30 W

SQL> CREATE INDEX idx_gr ON studenti(grupa);

Index created.
```

Cautati toti studentii din grupa Z si apoi pe toti din W… banuiti ce se intampla?

Statistici

- CBO utilizează statistici despre BD (de ex. privind: tabelele, coloanele, indecşii). De exemplu, pentru o **tabelă** poate memora:
  - valoarea maximă/minimă,
  - numărul de valori distincte,
  - numărul de câmpuri NULL,
  - distribuţia datelor (histograma),
  - dimensiunea tabelei (nr rânduri/blocuri).

Statistici

- CBO utilizeaza statistici despre BD (de ex. privind: tabelele, coloanele, indecşii). De exemplu, pentru un **index** poate memora:

  - adâncimea B-tree-ului,

  - numărul de frunze,

  - numărul de valori distincte din index,

  - factor de "*clustering*" (date situate pe aceeaşi pistă pe HDD sau în piste apropiate).

- **Utilizarea indecşilor nu e mereu soluţia cea mai potrivită.**

Indecşi bazaţi pe funcţii

Applies to
DB2 ✗
MySQL ✗
Oracle ✓
PostgreSQL ✓
SQL Server ✓

Funcţii

- Să presupunem că dorim să facem o căutare după **nume**.

```
---------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |     2 |   170 |     5   (0)| 00:00:01 |
|*  1 |  TABLE ACCESS FULL| STUDENTI |     2 |   170 |     5   (0)| 00:00:01 |
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

    1 - filter("NUME"='Popescu')
```

Funcţii

- Evident, această căutare va fi mai rapidă dacă:

```
SQL> CREATE INDEX idx_nume ON studenti(nume);
```

```
---------------------------------------------------------------------------
| Id  | Operation                    | Name     | Rows  | Bytes | Cost (%CPU)
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |          |     2 |   170 |     4   (0)
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI |     2 |   170 |     4   (0)
|*  2 |   INDEX RANGE SCAN           | IDX_NUME |     2 |       |     1   (0)
---------------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access("NUME"='Popescu')
```

Funcţii

- Ce se întamplă dacă vreau *ignorecase*?
- Pentru o astfel de cautare, deşi avem un index construit peste coloana cu `last_name`, acesta va fi ignorat [de ce ? – exemplu]

  [poate utilizarea unui alt *collation* ?!]*
- Pentru că BD nu cunoaşte rezultatul apelului unei funcţii a-priori, funcţia va trebui apelată pentru fiecare linie în parte.

*SQL Server sau MySQL nu fac distincţie între cases când sortează informaţiile în indecsi.

Funcţii

```
SQL> explain plan for select * from
         studenti where upper(nume)=upper('Popescu')

---------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          |    10 |   850 |     5   (0)| 00:0
|*  1 |  TABLE ACCESS FULL| STUDENTI |    10 |   850 |     5   (0)| 00:0
---------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter(UPPER("NUME")='POPESCU')
```

Işi dâ seama că e mai eficient să evalueze funcţia pentru valoarea constantă şi să nu facă acest lucru pentru fiecare rând în parte.

Funcţii

- Cum vede BD interogarea ?

```
SELECT * FROM studenti
   WHERE BLACKBOX(...) = 'POPESCU';
```

- Se observă totuşi că partea dreaptă a expresiei este evaluată o singură dată. În fapt filtrul a fost făcut pentru

  UPPER("nume")='POPESCU'

Funcţii

- Indexul va fi reconstruit peste **UPPER(nume)**

```
SQL> DROP INDEX idx_nume;

Index dropped.

SQL> CREATE INDEX idx_nume ON studenti(upper(nume));

Index created.
```

Functii - *function-based index* (**FBI**)

```
SQL> explain plan for select * from
         studenti where upper(nume)=upper('Popescu')
```

```
---------------------------------------------------------------------------
| Id  | Operation                    | Name     | Rows  | Bytes | Cost (%CP
---------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |          |    10 |   850 |     4   (
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI |    10 |   850 |     4   (
|*  2 |   INDEX RANGE SCAN           | IDX_NUME |     4 |       |     1   (
---------------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access(UPPER("NUME")='POPESCU')
```

Funcţii

- În loc să pună direct valoarea câmpului în index, un **FBI stochează valoarea returnată de funcţie**.
- Din acest motiv funcţia trebuie să returneze mereu aceeaşi valoare: nu sunt permise decât funcţii **deterministe**.
- Nu se pot construi FBI cu funcţii ce returnează valori aleatoare sau pentru cele care utilizează data sistemului pentru a calcula ceva.

  [e.g. days untill xmas]

Funcţii

- Nu există cuvinte rezervate sau optimizari pentru FBI (altele decât cele deja explicate).
- Uneori instrumentele pentru *Object relation mapping* (ORM tools) injectează din prima o funcţie de conversie a tipului literelor (upper / lower). De ex. Hibernate converteşte totul în lower.
- Puteţi construi proceduri stocate deterministe ca să fie folosite în FBI. getAge ?!?! – cum faceți să calculați care sunt angajații ce au 50 de ani ?

Funcţii – nu indexaţi TOT

- De ce ? (nu are sens să fac un index pt. *lower*) (dacă tot aveţi peste upper). De fapt, dacă există o funcţie bijectivă de la felul în care sunt indexate datele la felul în care vreţi să interogaţi baza de date, mai bine refaceţi interogarea – cu siguranţă este posibil !).

- Încercaţi să unificaţi căile de acces ce ar putea fi utilizate pentru mai multe interogări.
- E mai bine să puneţi indecşii peste datele originale decât dacă aplicaţi funcţii peste acestea.

Parametri dinamici

Parametri dinamici (*bind parameters, bind variables*)

- Sunt metode alternative de a trimite informaţii către baza de date.
- În locul scrierii informaţiilor direct în interogare, se folosesc construcţii de tipul ? şi :name (sau @name) iar datele adevărate sunt transmise din apelul API
- E “ok” să punem valorile direct în interogare dar abordarea parametrilor dinamici are unele avantaje:

Parametri dinamici (bind parameters, bind variables)

- Avantajele folosirii parametrilor dinamici:
  - Securitate [împiedică SQL injection]
  - Performanţa [obligă QO să folosească acelaşi plan de execuţie]

Parametri dinamici (bind parameters, bind variables)

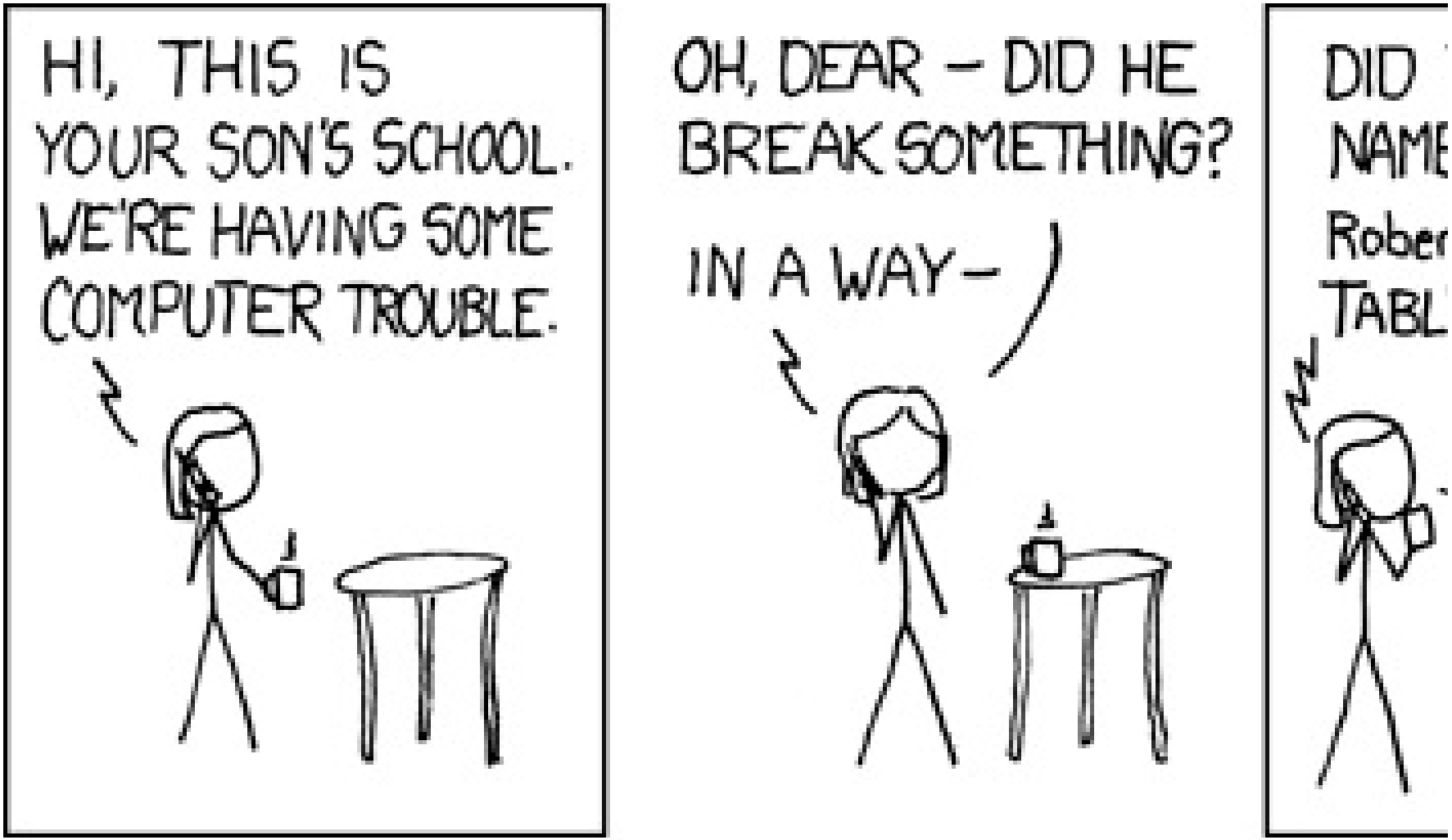
- Securitate: impiedică SQL injection*

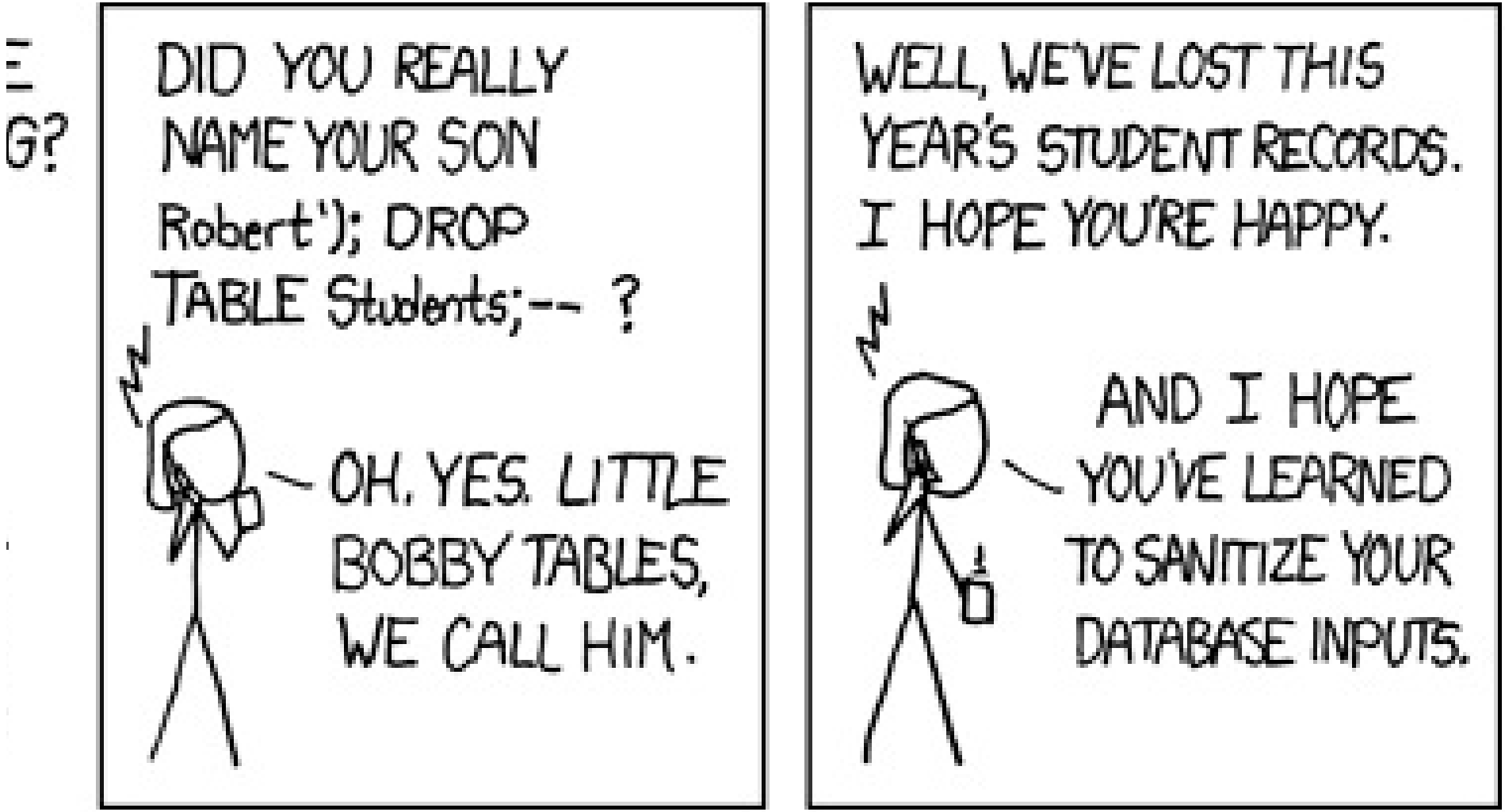
```
statement = "SELECT * FROM studenti
  WHERE nume ='" + userName + "';“
```

Dacă `userName` e modificat în `' or '1'='1`

Dacă `userName` e modificat în: `a';DROP TABLE users; SELECT * FROM userinfo WHERE 't' = 't`

* http://en.wikipedia.org/wiki/SQL_injection


HI, THIS IS YOUR SON'S SCHOOL. WE'RE HAVING SOME COMPUTER TROUBLE.
OH, DEAR – DID HE BREAK SOMETHING?
IN A WAY–
DID
NAME
Rober
TABL


DID YOU REALLY NAME YOUR SON Robert'); DROP TABLE Students;-- ?
OH. YES. LITTLE BOBBY TABLES, WE CALL HIM.
WELL, WE'VE LOST THIS YEAR'S STUDENT RECORDS. I HOPE YOU'RE HAPPY.
AND I HOPE YOU'VE LEARNED TO SANITIZE YOUR DATABASE INPUTS.

Parametri dinamici (bind parameters, bind variables)

- Avantajele folosirii parametrilor dinamici:
  - Securitate
  - Performanţă
- Performanţă: Baze de date (Oracle, SQL Server) pot salva (în cache) execuţii ale planurilor pe care le-au considerat eficiente dar **DOAR** dacă interogarile sunt **EXACT** la fel. Trimiţând valori diferite (nedinamic), sunt formulate interogari diferite.

Parametri dinamici (bind parameters, bind variables)

- Exemplu cu două interogări ce ar fi executate diferit când au distribuţia datelor diferită.

- Neavând efectiv valorile, se va executa planul care este considerat mai eficient dacă valorile date pentru **`câmpul interogat`** ar fi distribuite uniform. [atenţie, nu valorile din tabela ci cele din interogare !]

Parametri dinamici (bind parameters, bind variables)

- Query optimizer este ca un compilator:
  - daca îi sunt trecute valori ca şi constante, se foloseşte de ele în acest mod;
  - dacă valorile sunt dinamice, le vede ca variabile neiniţializate şi le foloseşte ca atare.
- Atunci de ce ar funcţiona mai bine când nu sunt ştiute valorile dinainte ?

Parametri dinamici (bind parameters, bind variables)

- Atunci când este trimisă valoarea, *The query optimizer* va construi **mai multe** planuri, va stabili care este cel mai bun după care îl va executa. În timpul ăsta, s-ar putea ca un plan (prestabilit), deşi mai puţin eficient, să fi executat deja interogarea.
- Utilizarea parametrilor dinamici e ca şi cum ai compila programul de fiecare dată.

Parametri dinamici (bind parameters, bind variables)

- Cine “bindeaza” variabilele poate face eficientă interogarea (programatorul): se vor folosi parametri dinamici pentru toate variabilele MAI PUTIN pentru cele pentru care se doreşte să influenţeze planului de execuţie.

- *In all reality, there are only a few cases in which the actual values affect the execution plan. You should therefore use bind parameters if in doubt—just to prevent SQL injections.*

Parametri dinamici (bind parameters, bind variables) – exemplu Java:

Fără bind parameters:

```
int valoare = 1200;
Statement command =
  connection.createStatement(
        "select nume, prenume" +
        " from studenti" +
        " where bursa = " +
     valoare );
```

Parametri dinamici (bind parameters, bind variables) – exemplu Java:

Cu bind parameters:

```
int valoare = 1200;
PreparedStatement command =
    connection.prepareStatement(
        "select nume, prenume" +
        " from studenti" +
        " where bursa = ?" );
command.setInt(1, valoare);
  int rowsAffected =
  preparedStatement.executeUpdate();
```

Se repetă pentru fiecare parametru

http://use-the-index-luke.com/sql/where-clause/bind-parameters - C#, PHP, Perl, Java, Ruby

Parametri dinamici (bind parameters, bind variables)- Ruby:

Fără parametri dinamici:

```
dbh.execute("select nume, prenume" +
 " from studenti" +
 " where bursa = #{valoare}");
```

Cu parametri dinamici:

```
dbh.prepare("select nume, prenume" +
   " from studenti" +
   " where bursa = ?");
dbh.execute(valoare);
```

Parametri dinamici (bind parameters, bind variables)

- Semnul întrebării indică o poziţie. El va fi indentificat prin 1,2,3… (poziţia lui) atunci când se vor trimite efectiv parametri.
- Se poate folosi “@id” (în loc de ? şi de 1,2…).

Parametri dinamici (bind parameters, bind variables)

- Parametri dinamici **nu pot schimba structura interogarii** (Ruby):

```
String sql = prepare("SELECT * FROM ?
  WHERE ?");

sql.execute(studenti',
                    'bursa = '1200');
```

Pentru a schimba structura interogarii: dynamic SQL.

Căutări pe intervale

Q: Dacă avem două coloane, una dintre ele cu foarte multe valori diferite şi cealaltă cu foarte multe valori identice. Pe care o punem prima în index ?

Q: Dacă avem două coloane, una dintre ele cu foarte multe valori diferite şi cealaltă cu foarte multe valori identice. Pe care o punem prima în index ?

[carte de telefon:numele sunt mai diversificate decat prenumele]

Căutări pe intervale

- Sunt realizate utilizând operatorii <, > sau folosind BETWEEN.
- Cea mai mare problemă a unei căutari într-un interval este *traversarea frunzelor.*
- Ar trebui ca intervalele să fie cât mai mici posibile. Intrebările pe care ni le punem:
  - *unde începe un index scan* ?
  - *unde se termină* ?

Căutări pe intervale

```
SELECT nume, prenume, data_nastere
FROM studenti
WHERE
  data_nastere >= TO_DATE(?, 'YYYY-MM-DD')
   AND
  data_nastere <= TO_DATE(?, 'YYYY-MM-DD')
```

Început

Sfârşit

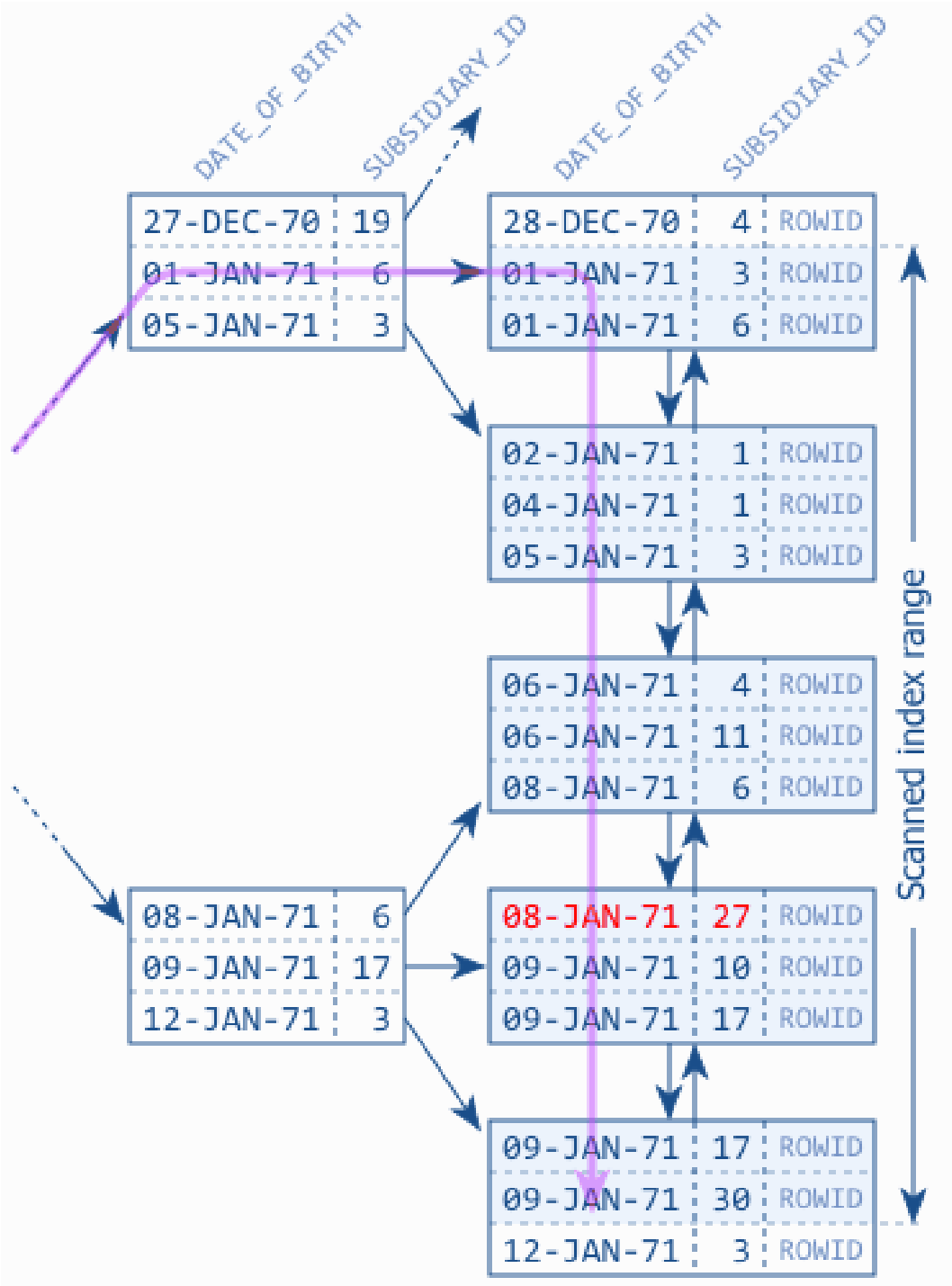
Căutări pe intervale

```
SELECT nume, prenume, data_nastere
FROM studenti
WHERE
  data_nastere >= TO_DATE(?, 'YYYY-MM-DD')
   AND
  data_nastere <= TO_DATE(?, 'YYYY-MM-DD')
AND grupa = ?
```

???

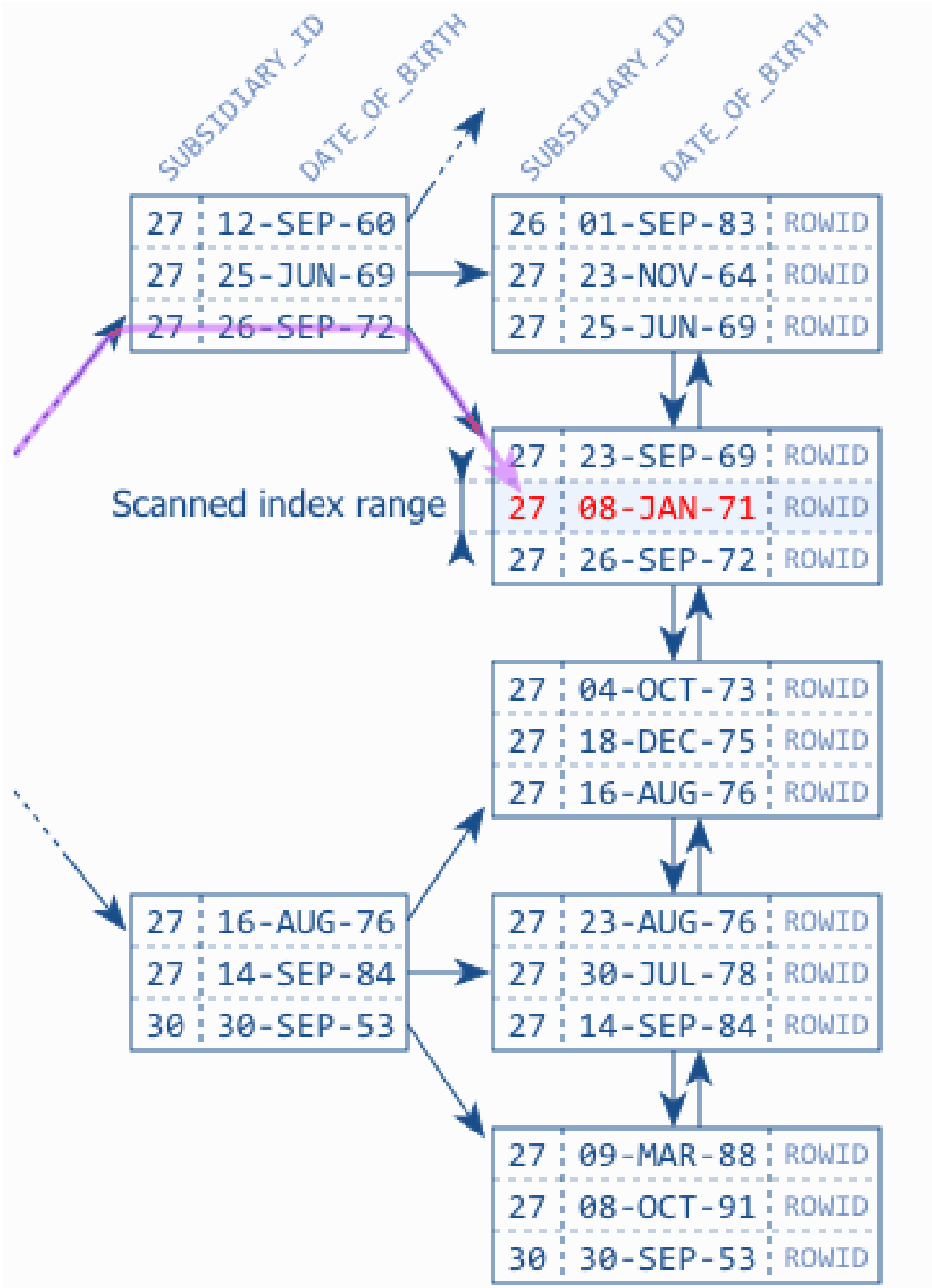
Căutări pe intervale

- Indexul ideal acoperă ambele coloane.
- În ce ordine ar fi cel mai bine să adăugăm coloanele în index ?



1 Ianuarie 1971
9 ianuarie 1971

Grupa = 27



Grupa = 27

1 Ianuarie 1971
9 ianuarie 1971

Căutări pe intervale

Regulă: indexul pentru egalitate primul şi apoi cel pentru interval !

Nu e neapărat bine ca să punem pe prima poziţie coloana cea mai diversificată.

Căutări pe intervale

- Depinde şi de ce interval căutăm (pentru intervale foarte mari s-ar putea sa fie mai eficient invers).
- Nu este neapărat ca acea coloana cu valorile cele mai diferite să fie prima în index – vezi cazul precedent în care sunt doar 30 de grupe si 365 de zile de nastere (x ani).
- Ambele indexări făceau *match* pe 13 înregistrări.

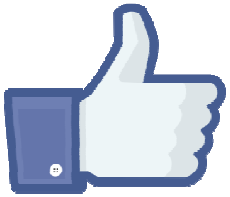
Cautari pe intervale

- Operatorul BETWEEN este echivalent cu o cautare in interval dar considerand si marginile intervalului.

```
DATE_OF_BIRTH BETWEEN '01-JAN-71'
   AND '10-JAN-71
```

Este echivalent cu:

```
DATE_OF_BIRTH >= '01-JAN-71' AND
  DATE_OF_BIRTH <= '10-JAN-71’
```



LIKE

LIKE

- Operatorul LIKE poate avea repercusiuni nedorite asupra interogarii (chiar când sunt folosiți indecși).
- Unele interogări în care este folosit LIKE se pot baza pe indecși, altele nu. Diferența o face poziția caracterului % .

LIKE

```
SQL> CREATE INDEX IDX_NUME ON STUDENTI(NUME);

Index created.

SQL> EXPLAIN PLAN FOR SELECT * FROM STUDENTI WHERE NUME LIKE 'Pop%escu'

Explained.

-----------------------------------------------------------------------
| Id  | Operation                   | Name     | Rows  | Bytes | Cost (%CP
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT            |          |     3 |   369 |     4   (
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI |     3 |   369 |     4   (
|*  2 |   INDEX RANGE SCAN          | IDX_NUME |     9 |       |     2   (
-----------------------------------------------------------------------

Predicate Information (identified by operation id):

  2 - access("NUME" LIKE 'Pop%escu')
      filter("NUME" LIKE 'Pop%escu')
```

LIKE

- Doar primele caractere dinainte de % pot fi utilizate în căutarea bazată pe indecși. Restul caracterelor sunt utilizate pentru a filtra rezultatele obținute.
- Cum ar fi procesate diverse interogări în funcție de poziția caracterului % (pentru numele Winand).

LIKE

LIKE 'WI%ND'	LIKE 'WIN%D'	LIKE 'WINA%'
WIAW	WIAW	WIAW
WIBLQQNPUA	WIBLQQNPUA	WIBLQQNPUA
WIBYHSNZ	WIBYHSNZ	WIBYHSNZ
WIFMDWUQMB	WIFMDWUQMB	WIFMDWUQMB
WIGLZX	WIGLZX	WIGLZX
WIH	WIH	WIH
WIHTFVZNLC	WIHTFVZNLC	WIHTFVZNLC
WIJYAXPP	WIJYAXPP	WIJYAXPP
WINAND	WINAND	WINAND
WINBKYDSKW	WINBKYDSKW	WINBKYDSKW
WIPOJ	WIPOJ	WIPOJ
WISRGPK	WISRGPK	WISRGPK
WITJIVQJ	WITJIVQJ	WITJIVQJ
WIW	WIW	WIW
WIWGPJMQGG	WIWGPJMQGG	WIWGPJMQGG
WIWKHLBJ	WIWKHLBJ	WIWKHLBJ
WIYETHN	WIYETHN	WIYETHN
WIYJ	WIYJ	WIYJ

LIKE

- Ce se intâmplă dacă LIKE-ul este de forma **`LIKE '%Po%escu'`** ?

LIKE

- A se evita expresii care încep cu %.
- În teorie, %, influențează felul în care este căutată expresia. În practică, **dacă sunt utilizați parametri dinamici**, nu se știe cum *Querry optimizer* va considera ca este mai bine să procedeze: ca și cum interogarea ar începe cu % sau ca și cum ar începe fără?

LIKE

- Dacă avem de căutat un **cuvânt într-un text**, nu contează dacă acel cuvânt este trimis ca parametru dinamic sau hardcodat în interogare. Căutarea va fi oricum de tipul %cuvant% . Totuși, folosind parametri dinamici, măcar evităm SQL injection.

LIKE

- Pentru a “optimiza” caăutările cu clauza LIKE, se poate utiliza în mod intenționat alt camp indexat (dacă se știe că intervalul ce va fi returnat de index va conține oricum textul ce conține parametrul din like).

Q: Cum ați putea indexa totuși pentru a optimiza o căutare care să aiba ca și clauză:

```
LIKE '%Popescu'
```

Indecşi Parţiali

Indexarea NULL

Applies to
DB2 ✗
MySQL ✗
Oracle ✗
PostgreSQL ✓
SQL Server ✓

- Să analizăm interogarea:

```
SELECT message
FROM messages
WHERE processed = 'N'
      AND receiver = ?
```

- Preia toate mailurile nevizualizate (de exemlu). Cum ați indexa ? [ambele sunt cu =]

- Am putea crea un index de forma:

```
CREATE INDEX messages_todo ON
  messages (receiver, processed)
```

- Se observă că `processed` împarte tabela în două categorii: mesaje procesate și mesaje neprocesate.

Indecși parțiali

- **Unele** BD permit indexarea parțiala. Asta înseamnă că indexul nu va fi creat decât peste anumite linii din tabel.

```
CREATE INDEX messages_todo
  ON messages (receiver)
  WHERE processed = 'N'
```

Atenție: nu merge în Oracle 11g XE

Indecși parțiali

- Ce se întâmplă la execuția codului:

```
SELECT message
    FROM messages
    WHERE processed = 'N';
```

Indecși parțiali

- Indexul nou construit este redus și pe verticală (pentru că are mai puține linii) dar și pe orizontală (nu mai trebuie să aibă grijă de coloana “processed”).
- Se poate întâmpla ca dimensiunea să fie constantă (de exemplu nu am mereu ~500 de mailuri necitite) chiar dacă numărul liniilor din BD crește.

NULL în Oracle

- Ce este NULL în Oracle ?
- În primul rând trebuie folosit “IS NULL” și nu “=NULL”.
- NULL nu este mereu conform standardului (ar trebui să însemne absența datelor).
- Oracle tratează un șir vid ca și NULL ?!?! (de fapt trateaza ca NULL orice nu știe sau nu înțelege sau care nu există).

```
   SELECT     '0 IS NULL???' AS "what is NULL?" FROM dual
    WHERE      0 IS NULL
UNION ALL
   SELECT    '0 is not null' FROM dual
    WHERE     0 IS NOT NULL
UNION ALL
   SELECT ''''' IS NULL???'  FROM dual
    WHERE    '' IS NULL
UNION ALL
   SELECT ''''' is not null' FROM dual
    WHERE    '' IS NOT NULL


what is NULL?
--------------
0 is not null
'' IS NULL???
```

NULL în Oracle

- Mai mult, Oracle tratează NULL ca șir vid:

```
SELECT dummy
     , dummy || ''
     , dummy || NULL
  FROM dual


D D D
- - -
X X X
```

NULL în Oracle

- Dacă am creat un index după o coloană X și apoi adăugăm o înregistrare care să aibă NULL pentru X, acea înregistrare nu este indexată.

NULL în Oracle

Neinserand data de nastere, aceasta va fi NULL

```
UPDATE STUDENTI SET DATA_NASTERE='' WHERE
  ID=100;
```

- Noul rând nu va fi indexat:

Table access full

```
SELECT nume, prenume
    FROM studenti
    WHERE data_nastere IS NULL
```

```
alter table studenti modify (data_nastere null);
```

Indexarea NULL în Oracle

```
CREATE INDEX demo_null ON studenti
  (id, data_nastere);
```

NOT NULL

- Și apoi:

```
SELECT nume, prenume
   FROM studenti
   WHERE id = 100
         AND data_nastere IS NULL
```

Indexarea NULL în Oracle

```
-----------------------------------------------------------------------
| Id  | Operation                    | Name      | Rows  | Bytes | Cost (%
-----------------------------------------------------------------------
|   0 | SELECT STATEMENT             |           |     1 |    48 |     1
|   1 |  TABLE ACCESS BY INDEX ROWID| STUDENTI  |     1 |    48 |     1
|*  2 |   INDEX RANGE SCAN           | DEMO_NULL |     1 |       |     1
-----------------------------------------------------------------------

Predicate Information (identified by operation id):

   2 - access("ID"=100 AND "DATA_NASTERE" IS NULL)
```

- Ambele predicate sunt utilizate !

Indexarea NULL în Oracle

- Atunci când indexăm după un câmp ce s-ar putea să fie NULL, pentru a ne asigura că și aceste rânduri sunt indexate, trebuie adăugat un câmp care să fie NOT NULL ! (poate fi adăugată și o constantă – de exemplu ‘1’):

```
DROP INDEX DEMO_NULL;
CREATE INDEX DEMO_NULL ON
  STUDENTI(DATA_NASTERE,'1');
```

Asta este NOT NULL

```
DROP INDEX emp_dob;
CREATE INDEX emp_dob_name
          ON employees (date_of_birth, last_name);


SELECT *
  FROM employees
 WHERE date_of_birth IS NULL


-----------------------------------------------------------------
|Id |Operation                    | Name          | Rows | Cost |
-----------------------------------------------------------------
| 0 |SELECT STATEMENT             |               |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES     |    1 |    3 |
|*2 |  INDEX RANGE SCAN           | EMP_DOB_NAME  |    1 |    2 |
-----------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("DATE_OF_BIRTH" IS NULL)
```

```
ALTER TABLE employees MODIFY last_name NULL;


SELECT *
  FROM employees
 WHERE date_of_birth IS NULL


-----------------------------------------------------
| Id | Operation          | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

- Fără NOT NULL pus pe last_name (care e folosit în index), indexul este inutilizabil.
- Se “gândește” că poate există cazul când ambele câmpuri sunt nule și acel caz nu e băgat în index.

Indexarea NULL în Oracle

- O funcție creată de utilizator este considerată ca fiind NULL (indiferent dacă este sau nu).

- Există anumite funcții din Oracle care sunt recunoscute că întorc NULL atunci când datele de intrare sunt NULL (de exemplu funcția upper).

```
CREATE OR REPLACE FUNCTION blackbox(id IN NUMBER) RETURN NUMBER
DETERMINISTIC
IS BEGIN
   RETURN id;
END;

DROP INDEX emp_dob_name;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, blackbox(employee_id));


SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------
| Id | Operation          | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

In opinia lui, ambele pot fi NULL.

Desi id este NOT NULL

Ii spunem clar ca nu ne intereseaza unde functia da NULL.

```
SELECT *
  FROM employees
 WHERE date_of_birth IS NULL
   AND blackbox(employee_id) IS NOT NULL;

-----------------------------------------------------------
|Id |Operation                      | Name       | Rows | Cost |
-----------------------------------------------------------
| 0 |SELECT STATEMENT               |            |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES  |    1 |    3 |
|*2 |  INDEX RANGE SCAN             | EMP_DOB_BB |    1 |    2 |
-----------------------------------------------------------
```

```
ALTER TABLE employees ADD bb_expression
      GENERATED ALWAYS AS (blackbox(employee_id)) NOT NULL;

DROP   INDEX emp_dob_bb;
CREATE INDEX emp_dob_bb
    ON employees (date_of_birth, bb_expression);


SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;
```

Si folosim coloana in index

Sau ii spunem ca acest camp este mereu NOT NULL.

```
-----------------------------------------------------------
|Id |Operation                        | Name        | Rows | Cost |
-----------------------------------------------------------
| 0 |SELECT STATEMENT                 |             |    1 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES   |    1 |    3 |
|*2 |  INDEX RANGE SCAN               | EMP_DOB_BB  |    1 |    2 |
-----------------------------------------------------------
```

```
DROP INDEX emp_dob_bb;

CREATE INDEX emp_dob_upname
    ON employees (date_of_birth, upper(last_name));

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------------
|Id |Operation                      | Name            | Cost |
-----------------------------------------------------------
| 0 |SELECT STATEMENT               |                 |    3 |
| 1 | TABLE ACCESS BY INDEX ROWID| EMPLOYEES       |    3 |
|*2 |  INDEX RANGE SCAN             | EMP_DOB_UPNAME  |    2 |
-----------------------------------------------------------
```

Daca initial last_name este nenul va sti ca upper(last_name) este tot nenul.

```
ALTER TABLE employees MODIFY last_name NULL;

SELECT *
  FROM employees
 WHERE date_of_birth IS NULL;

-----------------------------------------------------
| Id | Operation          | Name      | Rows | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |           |    1 |  477 |
|* 1 |  TABLE ACCESS FULL| EMPLOYEES |    1 |  477 |
-----------------------------------------------------
```

Emularea indecșilor parțiali în Oracle

```
CREATE INDEX messages_todo
   ON messages (receiver)
   WHERE processed = 'N'
```

- Avem nevoie de o funcție care să returneze NULL de fiecare dată când mesajul a fost procesat.

Emularea indecșilor parțiali în Oracle

```
CREATE OR REPLACE FUNCTION
  pi_processed(processed CHAR,
  receiver NUMBER)
RETURN NUMBER DETERMINISTIC AS
BEGIN
      IF processed IN ('N')
            THEN RETURN receiver;
            ELSE RETURN NULL;
       END IF;
END; /
```

Pentru a putea fi utilizata in index.

```
CREATE INDEX messages_todo
          ON messages (pi_processed(processed, receiver));

SELECT message
  FROM messages
 WHERE pi_processed(processed, receiver) = ?
```

Deoarece stie ca aici va veni o valoare, QO face un singur plan (cu index). Daca ar fi fost null ar fi fost testat cu “IS NULL”.

```
-----------------------------------------------------------
|Id | Operation                    | Name          | Cost |
-----------------------------------------------------------
| 0 | SELECT STATEMENT             |               | 5330 |
| 1 |  TABLE ACCESS BY INDEX ROWID| MESSAGES      | 5330 |
|*2 |   INDEX RANGE SCAN           | MESSAGES_TODO | 5303 |
-----------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   2 - access("PI_PROCESSED"("PROCESSED","RECEIVER")=:X)
```

Condiții *obfuscate*

Obfuscat = ascuns intenționat, special pentru ca altcineva să nu poată înțelege codul

Metode de Obfuscare – șiruri numerice

- Sunt numere memorate în coloane de tip text
- Deși nu e practic, un index poate fi folosit peste un astfel de șir de caractere (indexul este peste șirul de caractere):

```
SELECT ... FROM ... WHERE
  numeric_string = '42'
```

- Dacă s-ar face o căutare de genul:

Metode de Obfuscare – șiruri numerice

```
SELECT ... FROM ... WHERE
  numeric_string = 42
```

- Unele SGBDuri vor semnala o eroare (PostgreSQL) în timp ce altel vor face o conversie astfel:

```
SELECT ... FROM ... WHERE
  TO_NUMBER(numeric_string) = 42
```

Va merge pe index ? (care era construit peste sirul de caractere ?!?!)

Metode de Obfuscare – șiruri numerice

- Problema este că nu ar trebui să convertim șirul de caractere din tabel ci mai degrabă să convertim numărul (pentru că indexul e pe șir):

```
SELECT ... FROM ... WHERE
        numeric_string = TO_CHAR(42)
```

- De ce nu face baza de date conversia în acest mod ? Pentru ca datele din tabel ar putea fi stocate ca ‘42’ dar si ca ‘042’, ‘0042’ care sunt diferite ca si siruri de caractere dar reprezinta acelasi numar.

Metode de Obfuscare – șiruri numerice

- Conversia se face din șiruri în numere deoarece ‘42’ sau ‘042’ vor avea aceeași valoare când sunt convertite. Totuși 42 nu va putea fi văzut ca fiind atât ‘42’ cât și ‘042’ când este convertit în șir numeric.
- Diferența nu este numai una de performanță dar chiar una ce ține de semantică.

Metode de Obfuscare – șiruri numerice

- Utilizarea șirurilor numerice într-o tabelă este problematică (de exemplu din cauză că poate fi stocat și altceva decât un număr).
- Regulă: folosiți tipuri de date numerice ca să stocați numere.

Metode de Obfuscare - Date

Dar intâi…

to_char vs to_date

Metode de Obfuscare - Date

- Data include o componentă temporală
- Trunc(DATE) setează data la miezul nopții.

```
SELECT ... FROM sales WHERE
  TRUNC(sale_date) =
    TRUNC(sysdate - INTERVAL '1' DAY)
```

Nu va merge corect dacă indexul este pus pe sale_date deoarece TRUNC=*blackBox*.

```
CREATE INDEX index_name ON table_name
  (TRUNC(sale_date))
```

Metode de Obfuscare - Date

- Este bine ca indecșii să îi punem peste datele originale (și nu peste funcții).
- Dacă facem acest lucru putem folosi același index și pentru căutări ale vânzărilor de ieri dar și pentru căutări a vânzărilor din ultima **oră** / săptămână / lună sau din luna N.

Metode de Obfuscare - Date

```
SELECT ... FROM sales WHERE
  DATE_FORMAT(sale_date, '%Y-%M') =
  DATE_FORMAT(now() , '%Y-%M')
```

- Caută vânzările din luna curentă. Mai rapid este:

```
SELECT ... FROM sales WHERE
  sale_date BETWEEN month_begin(?)
  AND month_end(?)
```

Metode de Obfuscare - Date

- Regulă: scrieți interogările pentru perioada ca și condiții explicite (chiar dacă e vorba de o singură zi).

```
sale_date >= TRUNC(sysdate) AND
  sale_date < TRUNC(sysdate + 1)
```

Metode de Obfuscare - Date

- O altă problemă apare la compararea tipurilor **date** cu **șiruri de caractere**:

```
SELECT ... FROM sales WHERE
  TO_CHAR(sale_Date, 'YYYY-MM-DD') = '1970-
  01-01'
```

- Problema este (iarași) conversia coloanei ce reprezintă data. Mai bine convertiți șirul în dată decât invers !
- Oamenii trăiesc cu impresia că parametrii dinamici trebuie să fie numere sau caractere. In fapt ele pot fi chiar și de tipul `java.util.Date`

Metode de Obfuscare - Date

- Dacă nu puteți trimite chiar un obiect de tip Date ca parametru, măcar nu faceți conversia coloanei (evitând a utiliza indexul). Mai bine:

Index peste sale_date

```
SELECT ... FROM sales WHERE sale_date
  = TO_DATE('1970-01-01', 'YYYY-MM-
  DD')
```

Fie direct sir de caractere sau chiar parametru dinamic trimis ca sir de caractere.

Metode de Obfuscare - Date

- Când sale_date conține o dată de tip timp, e mai bine să utilizăm intervale) :

```
SELECT ... FROM sales WHERE
   sale_date >= TO_DATE('1970-01-01',
 'YYYY-MM-DD') AND
   sale_date < TO_DATE('1970-01-01',
 'YYYY-MM-DD') + INTERVAL '1' DAY
```

```
sale_date LIKE SYSDATE
```

Metode de Obfuscare - Math

- Putem crea un index pentru ca urmatoarea interogare să funcționeze corect?

```
SELECT numeric_number FROM table_name
WHERE numeric_number - 1000 > ?
```

- Dar pentru:

```
SELECT a, b FROM table_name
WHERE 3*a + 5 = b
```

Metode de Obfuscare - Math

- În mod normal **NU** este bine să punem SGBD-ul să rezolve ecuații.Pentru el, și următoarea interogare va face full scan:

```
SELECT numeric_number FROM table_name
WHERE numeric_number + 0 > ?
```

Chiar de are index peste numeric_number, nu are peste suma lui cu 0 !

- Totuși am putea indexa în felul următor:

```
CREATE INDEX math ON table_name (3*a - b)
SELECT a, b FROM table_name
WHERE 3*a - b = -5;
```

Metode de Obfuscare – “Smart logic”

```
SELECT first_name, last_name,
  subsidiary_id, employee_id FROM
  employees WHERE
( subsidiary_id = :sub_id OR :sub_id
  IS NULL ) AND
( employee_id = :emp_id OR :emp_id IS
  NULL ) AND
( UPPER(last_name) = :name OR :name
  IS NULL )
```

Metode de Obfuscare – “Smart logic”

- Când nu se dorește utilizarea unuia dintre filtre, se trimite NULL în parametrul dinamic.
- Baza de date nu știe care dintre filtre este NULL și din acest motiv se așteaptă că toate pot fi NULL => TABLE ACCESS FULL + filtru (chiar dacă există indecși).
- Problema este că QO trebuie să găsească planul de execuție care să acopere **toate cazurile** (inclusiv când toți sunt NULL), pentru că va folosi același plan pentru toate interogarile cu variabile dinamice.

Metode de Obfuscare – “Smart logic”

- Soluția este să îi zicem BD ce avem nevoie și atât:

```
SELECT first_name, last_name,
  subsidiary_id, employee_id FROM
  employees
WHERE UPPER(last_name) = :name
```

- Problema apare din cauza *share execution plan* pentru parametri dinamici.

Performanța - Volumul de date

Don't ask a DBA to help you move furniture.
They've been known to drop tables…

Volumul de date

- O interogare devine mai lentă cu cât sunt mai multe date în baza de date

- Cât de mare este impactul asupra performanței dacă volumul datelor se dublează ?
- Cum putem îmbunătăți ?

Volumul de date

- Interogarea analizată:

```
SELECT count(*) FROM scale_data
  WHERE section = ? AND id2 = ?
```

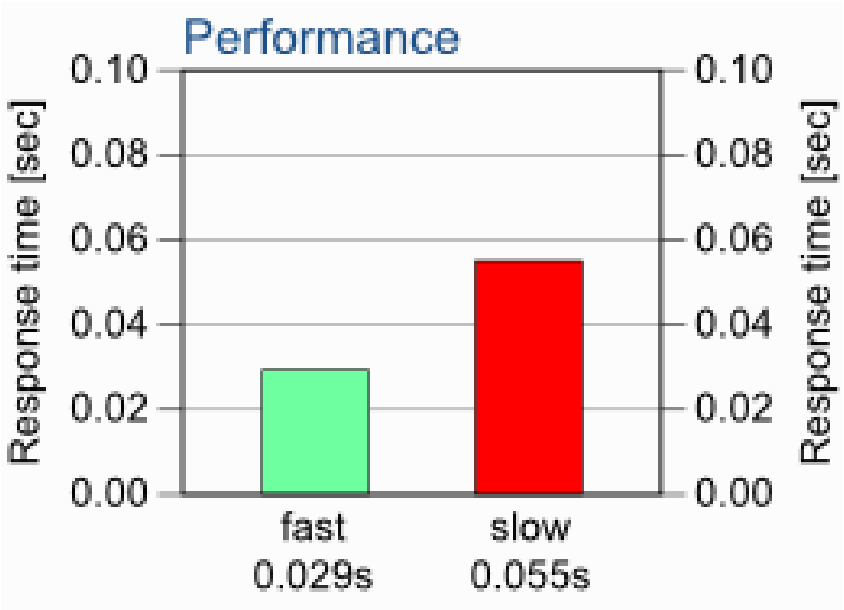
- Section are rolul de a controla volumul de date. Cu cât este mai mare section, cu atât este mai mare volumul de date returnat.
- Considerăm doi indecși: index1 și index2

Volumul de date

- Interogarea analizată:

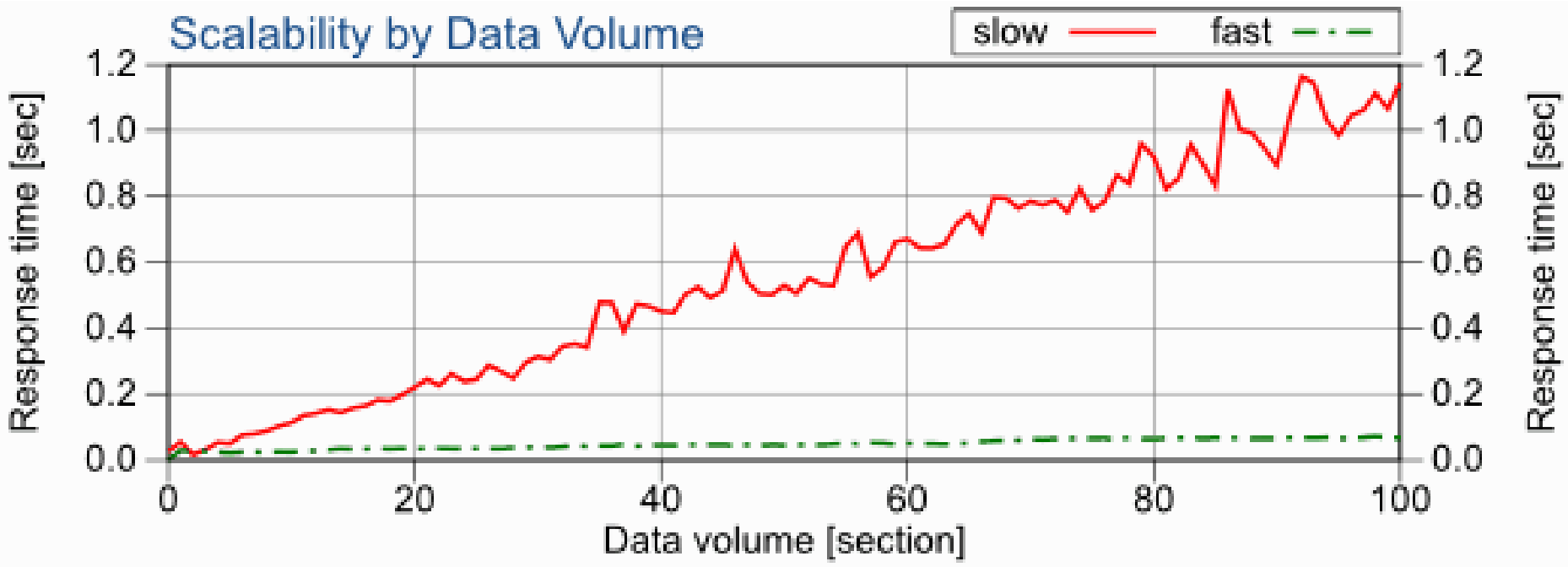
```
SELECT count(*) FROM scale_data
  WHERE section = ? AND id2 = ?
```

- *Section mic* – index1 și apoi index2



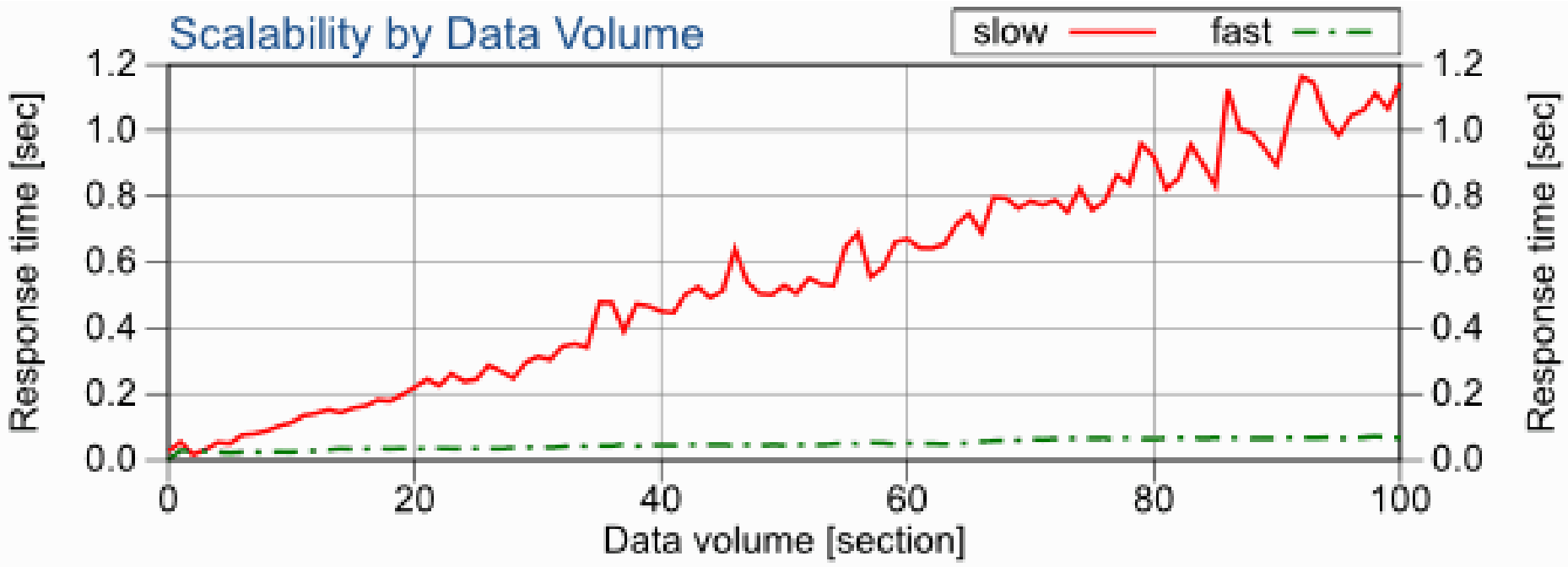
Volumul de date

- **Scalabilitatea** indică dependența performanței în funcție de factori precum **volumul de informații.**



Volumul de date

- index1 – timp dublu față de cel inițial
- index2 – trimp x20 față de cel inițial



Volumul de date

- Răspunsul unei interogări depinde de mai mulți factori. Volumul de date e unul dintre ei.
- Dacă o interogare merge bine în faza de test, nu e neapărat ca ea să funcționeze bine și în producție.
- Care este motivul pentru care apare diferența dintre index1 și index2 ?

Ambele par identice ca execuție:

```
| Id | Operation          | Name        | Rows  | Cost |
-----------------------------------------------------------
|  0 | SELECT STATEMENT   |             |     1 |  972 |
|  1 |  SORT AGGREGATE    |             |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW  |  3000 |  972 |
-----------------------------------------------------------
```

```
-----------------------------------------------------------
| Id   Operation          | Name        | Rows  | Cost |
-----------------------------------------------------------
|  0 | SELECT STATEMENT   |             |     1 |   13 |
|  1 |  SORT AGGREGATE    |             |     1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST  |  3000 |   13 |
-----------------------------------------------------------
```

Volumul de date

- Ce influențează un index ?
  - table acces
  - scanarea unui interval mare
- Nici unul din planuri nu indică acces pe baza indexului (`TABLE ACCES BY INDEX ROW ID`)
- Unul din intervale este mai mare atunci cand e parcurs…. trebuie să avem acces la "*predicate information*" ca să vedem de ce:

```
---------------------------------------------------------
| Id | Operation          | Name        | Rows   | Cost |
---------------------------------------------------------
|  0 | SELECT STATEMENT   |             |      1 |  972 |
|  1 |  SORT AGGREGATE    |             |      1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_SLOW  |   3000 |  972 |
---------------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A))
       filter("ID2"=TO_NUMBER(:B))
```

```
-----------------------------------------------------
| Id   Operation          | Name        | Rows   | Cost |
-----------------------------------------------------
|  0 | SELECT STATEMENT   |             |      1 |   13 |
|  1 |  SORT AGGREGATE    |             |      1 |      |
|* 2 |   INDEX RANGE SCAN| SCALE_FAST  |   3000 |   13 |
-----------------------------------------------------

Predicate Information (identified by operation id):
   2 - access("SECTION"=TO_NUMBER(:A) AND "ID2"=TO_NUMBER(:B))
```

Volumul de date

- Puteți spune cum a fost construit indexul având planurile de execuție ?

Volumul de date

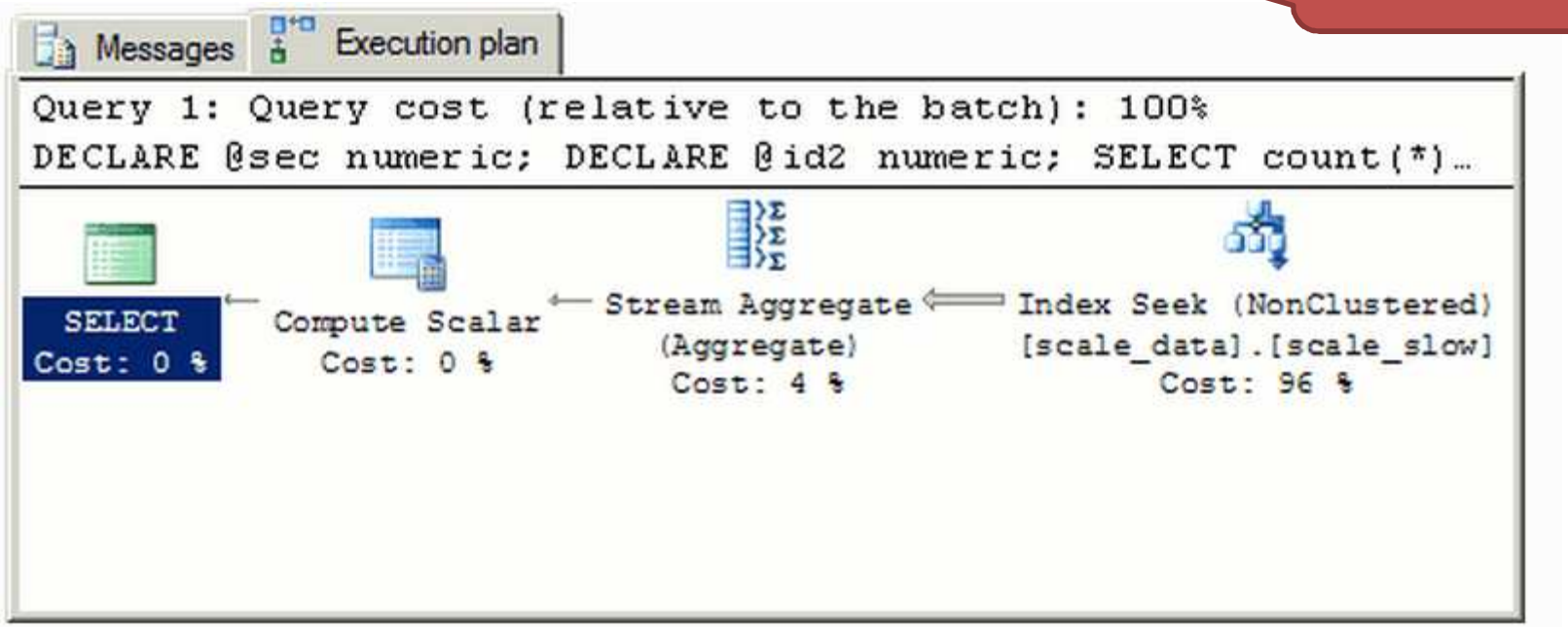
- Puteți spune cum a fost construit indexul având planurile de execuție ?
- `CREATE INDEX scale_slow ON scale_data (section, id1, id2);`
- `CREATE INDEX scale_fast ON scale_data (section, id2, id1);`

Campul id1 este adaugat doar pentru a pastra aceeasi dimensiune (sa nu se creada ca indexul *scale_fast* e mai rapid pentru ca are mai putine campuri in el).

Încărcarea sistemului

- Faptul ca am definit un index pe care îl considerăm bun pentru interogările noastre nu îl face să fie neapărat folosit de QO.
- *SQL Server Management Studio*

Arata predicatul doar ca un tooltip

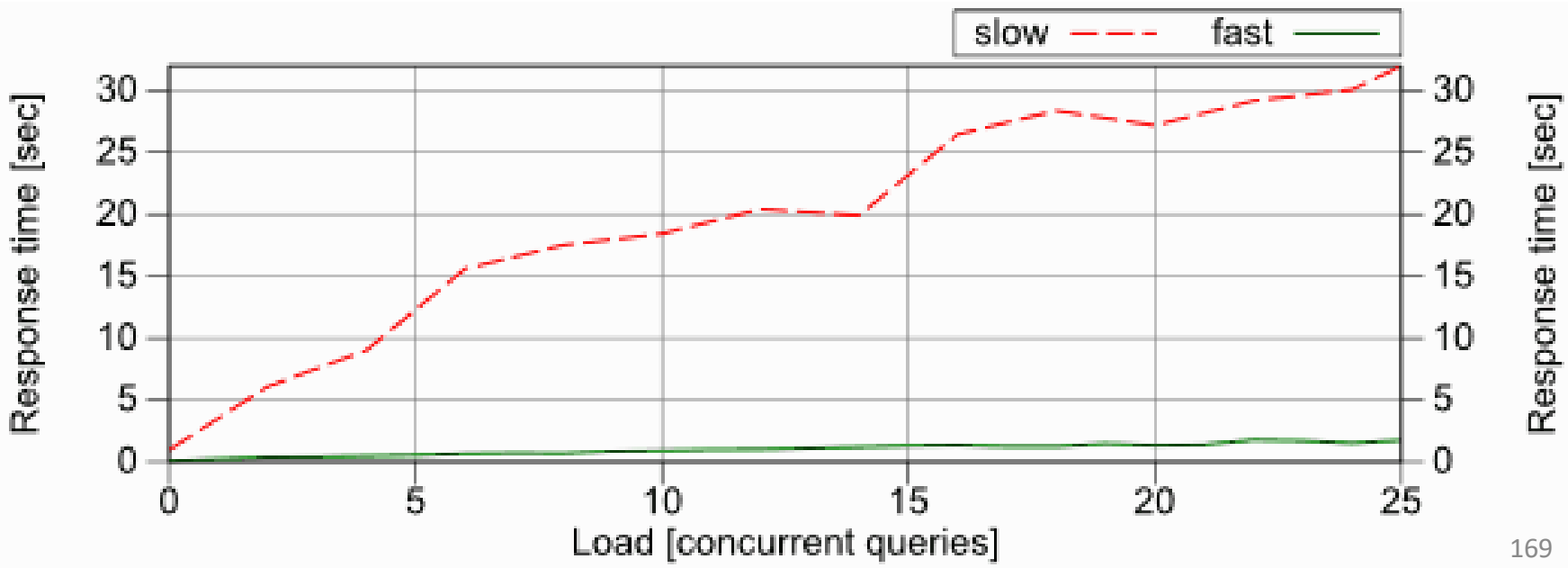


Încărcarea sistemului

- De regulă, împreună cu numărul de înregistrări, crește și numărul de accesări.
- **Numărul de accesari** este alt parametru ce intră în calculul scalabilității.

Încărcarea sistemului

- Dacă inițial era doar o singură accesare, considerând același scenariu dar cu 1-25 interogări concurente, timpul de raspuns crește:



Încărcarea sistemului

- Asta înseamnă că și dacă avem toata baza de date din producție și testăm totul pe ea, tot sunt șanse ca în realitate, din cauza numărului mare de interogări, să meargă mult mai greu.

- **Notă:** atenția dată planului de execuție este mai importantă decât *benchamarkuri* superficiale *(* gen *SQL Server Management Studio)*.

Încărcarea sistemului

- Ne-am putea aștepta ca **hardwareul mai puternic din producție să ducă mai bine sistemul**. În fapt, în faza de development nu există deloc latență – ceea ce nu se întâmplă în producție (unde accesul poate fi întârziat din cauza rețelei).
- http://blog.fatalmind.com/2009/12/22/latency-security-vs-performance/
- http://jamesgolick.com/2010/10/27/we-are-experiencing-too-much-load-lets-add-a-new-server..html

Timpi de răspuns + *throughput*

- Hardware mai performant nu este mai rapid doar poate duce mai multa încărcare. highway (daca adaug 10 benzi, nu inseamna ca si masinile vor merge de 10 ori mai rapid)
- Procesoarele single-core vs procesoarele multi-core (când e vorba de un singur task).
- Scalarea pe orizontală (adăugarea de procesoare) are același efect.
- Pentru a îmbunătăți timpul de răspuns este necesar un arbore eficient (chiar și în NoSQL).

Timpi de răspuns

- Indexarea corectă fac căutarea într-un B-tree în timp logaritmic.
- Sistemele bazate pe NoSQL par să fi rezolvat problema performanței prin scalare pe orizontală [analogie cu indecșii parțiali în care fiecare partiție este stocată pe o mașina diferită].
- Această scalabilitate este totuși limitată la operațiile de scriere într-un model denumit "eventual consistency" [Consistency / Availability / Partition tolerance = CAP theorem] http://en.wikipedia.org/wiki/CAP_theorem

Timpi de răspuns

- Mai mult hardware de obicei nu îmbunătățește sistemul.
- *Latency* al HDD [problema apare cand datele sunt culese din locații diferite ale HDDului – de exemplu în cadrul unei operații JOIN]. SSD?

"Facts"

- *Performance has two dimensions: response time and throughput.*
- *More hardware will typically not improve query response time.*
- *Proper indexing is the best way to improve query response time.*

Join

An SQL query walks into a bar and sees two tables.

He walks up to them and asks 'Can I join you?'

— Source: Unknown

Join

- *Join-ul* transforma datele dintr-un **model normalizat** intr-unul **denormalizat** care serveste unui anumit scop.
- Sensibil la latente ale discului (si fragmentare).

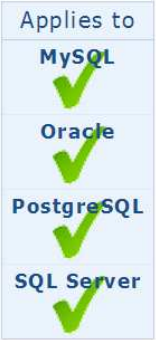
Join

- Reducerea timpilor = indexarea corecta ☺
- Toti algoritmii de join proceseaza doar doua tabele simultan (apoi rezultatul cu a treia, etc).
- Rezultatele de la un join sunt trimise in urmatoarea operatie join fara a fi stocate.
- Ordinea in care se efectueaza JOIN-ul influenteaza viteza de raspuns.[10, 30, 5, 60]
- QO incearca toate permutarile de JOIN.
- Cu cat sunt mai multe tabele, cu atat mai multe planuri de evaluat. [cate ?]

Join

- Cu cat sunt mai multe tabele, cu atat mai multe planuri de evaluat = O(n!)
- Nu este o problema cand sunt utilizati parametri dinamici [De ce ?]

Join – Nested Loops (anti patern)



- Ca si cum ar fi doua interogari: cea exterioara pentru a obtine o serie de rezultate dintr-o tabela si cea interioara ce preia fiecare rand obtinut si apoi informatia corespondenta din cea de-a doua tabela.
- Se pot folosi *Nested Selects* pentru a simula algoritmul de nested loops [latenta retelei, usurinta implementarii, Object-relational mapping (N+1 selects)].

Join – nested selects [PHP] java, perl on “luke…”

```
$qb = $em->createQueryBuilder();
$qb->select('e')
   ->from('Employees', 'e')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();

foreach ($r as $row) {
   // process Employee
   foreach ($row->getSales() as $sale) {
      // process Sale for Employee
   }
}
```

Join – nested selects

Doctrine

Only on source code level—don't forget to disable this for production. Consider building your own configurable logger.

```
$logger = new \Doctrine\DBAL\Logging\EchoSqlLogger;
$config->setSQLLogger($logger);
```

Join – nested selects

Doctrine 2.0.5 generates N+1 `select` queries:

```
SELECT e0_.employee_id AS employee_id0 -- MORE COLUMNS
  FROM employees e0_
 WHERE UPPER(e0_.last_name) LIKE ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?

   SELECT t0.sale_id AS SALE_ID1 -- MORE COLUMNS
     FROM sales t0
    WHERE t0.subsidiary_id = ?
      AND t0.employee_id = ?
```

Ce indecsi ati crea ca sa fie mai rapida executia ?

Join – nested selects

- DB executa joinul exact ca si in exemplul anterior. Indexarea pentru nested loops este similara cu cea din selecturile anterioare:

1. Un FBI (function based Index) peste UPPER(last_name)
2. Un Index concatenat peste subsidiary_id, employee_id

Join – nested selects

- Totusi, in BD nu avem latenta din retea.
- Totusi, in BD nu sunt transferate datele intermediare (care sunt *piped* in BD).

- Pont: executati JOIN-urile in baza de date si nu in Java/PHP/Perl sau in alt limbaj (ORM).

There you go: PLSQL style ;)

Join – nested selects

- Cele mai multe ORM permit SQL joins.
- *eager fetching* – probabil cel mai important (va prelua si tabela vanzari –in mod join– atunci cand se interogheaza angajatii).
- Totusi *eager fetching* nu este bun atunci cand este nevoie doar de tabela cu angajati (aduce si date irelevante) – nu am nevoie de vanzari pentru a face o carte de telefoane cu angajatii.
- O configurare statica nu este o solutie buna.

```
$qb = $em->createQueryBuilder();
$qb->select('e,s')
   ->from('Employees', 'e')
   ->leftJoin('e.sales', 's')
   ->where("upper(e.last_name) like :last_name")
   ->setParameter('last_name', 'WIN%');
$r = $qb->getQuery()->getResult();
```

Doctrine 2.0.5 generates the following SQL statement:

```
SELECT e0_.employee_id AS employee_id0
       -- MORE COLUMNS
  FROM employees e0_
  LEFT JOIN sales s1_
         ON e0_.subsidiary_id = s1_.subsidiary_id
        AND e0_.employee_id = s1_.employee_id
 WHERE UPPER(e0_.last_name) LIKE ?
```

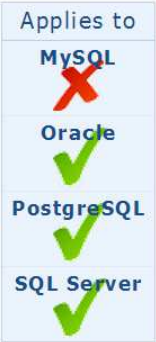
```
---------------------------------------------------------------
|Id |Operation                        | Name         | Rows | Cost |
---------------------------------------------------------------
| 0 |SELECT STATEMENT                 |              |  822 |   38 |
| 1 | NESTED LOOPS OUTER              |              |  822 |   38 |
| 2 |  TABLE ACCESS BY INDEX ROWID| EMPLOYEES    |    1 |    4 |
|*3 |   INDEX RANGE SCAN              | EMP_UP_NAME  |    1 |      |
| 4 |  TABLE ACCESS BY INDEX ROWID| SALES        |  821 |   34 |
|*5 |   INDEX RANGE SCAN              | SALES_EMP    |   31 |      |
---------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
  3 - access(UPPER("LAST_NAME") LIKE 'WIN%')
      filter(UPPER("LAST_NAME") LIKE 'WIN%')
  5 - access("E0_"."SUBSIDIARY_ID"="S1_"."SUBSIDIARY_ID"(+)
        AND  "E0_"."EMPLOYEE_ID"  ="S1_"."EMPLOYEE_ID"(+))
```

Join – nested selects

- Sunt bune daca sunt intoarse un numar mic de inregistrari.

- http://blog.fatalmind.com/2009/12/22/latency-security-vs-performance/

Join – Hash join



- Evita traversarea multipla a B-tree din cadrul inner-querry (din nested loops) construind cate o tabela hash pentru inregistrarile candidat.
- *Hash join* imbunatatit daca sunt selectate mai putine coloane.
- A se indexa predicatele independente din where pentru a imbunatati performanta. (pe ele este construit hashul)

Join – Hash join

```
SELECT * FROM
sales s JOIN employees e
ON (s.subsidiary_id = e.subsidiary_id
  AND s.employee_id = e.employee_id )
WHERE s.sale_date > trunc(sysdate) -
  INTERVAL '6' MONTH
```

Join – Hash join

```
---------------------------------------------------------------
| Id | Operation           | Name      | Rows  | Bytes | Cost |
---------------------------------------------------------------
|  0 | SELECT STATEMENT    |           | 49244 |    59M| 12049|
|* 1 |  HASH JOIN          |           | 49244 |    59M| 12049|
|  2 |   TABLE ACCESS FULL| EMPLOYEES | 10000 |     9M|   478|
|* 3 |   TABLE ACCESS FULL| SALES     | 49244 |    10M| 10521|
---------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
          AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID")
   3 - filter("S"."SALE_DATE">TRUNC(SYSDATE@!)
                              -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

Join – Hash join

- **Indexarea predicatelor utilizate in join nu imbunatatesc performanta hash join !!!**
- Un index ce ar putea fi utilizat este peste sale_date
- Cum ar arata daca s-ar utiliza indexul ?

Join – Hash join

```
---------------------------------------------------------------
| Id | Operation                      | Name       | Bytes| Cost|
---------------------------------------------------------------
|  0 | SELECT STATEMENT               |            |   59M| 3252|
|* 1 |  HASH JOIN                     |            |   59M| 3252|
|  2 |   TABLE ACCESS FULL            | EMPLOYEES  |    9M|  478|
|  3 |   TABLE ACCESS BY INDEX ROWID| SALES      |   10M| 1724|
|* 4 |    INDEX RANGE SCAN            | SALES_DATE|      |      |
---------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
   1 - access("S"."SUBSIDIARY_ID"="E"."SUBSIDIARY_ID"
          AND "S"."EMPLOYEE_ID"  ="E"."EMPLOYEE_ID"  )
   4 - access("S"."SALE_DATE" > TRUNC(SYSDATE@!)
                          -INTERVAL'+00-06' YEAR(2) TO MONTH)
```

Join – Hash join

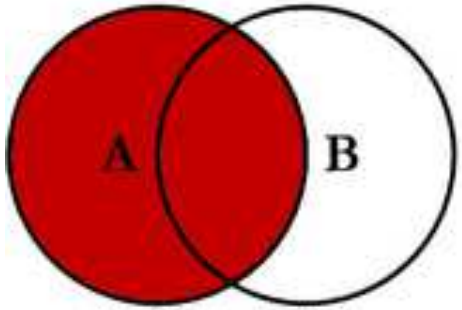
- Ordinea conditiilor din join nu influenteaza viteza (la nested loops influenta).

Bibliografie (online)

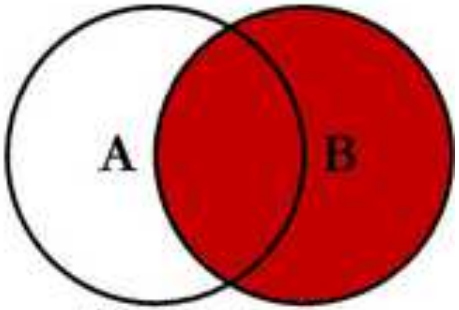
- http://use-the-index-luke.com/

  (puteti cumpara si cartea in format PDF – dar nu contine altceva decat ceea ce este pe site)

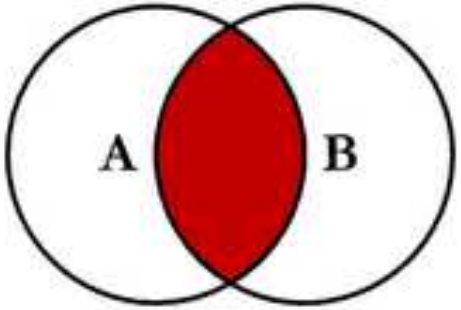
SQL JOINS



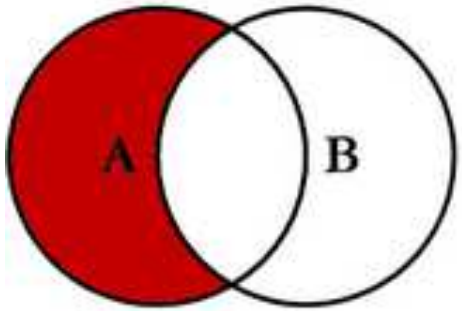

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



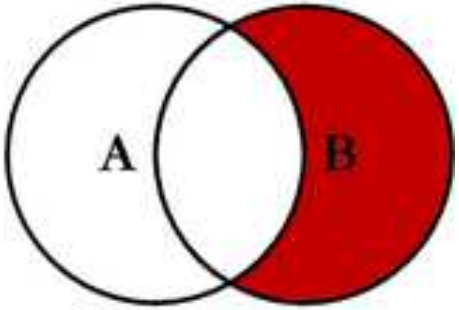

```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



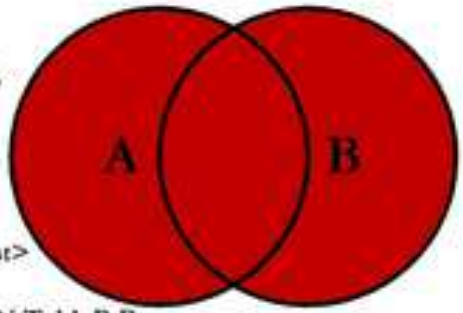

```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



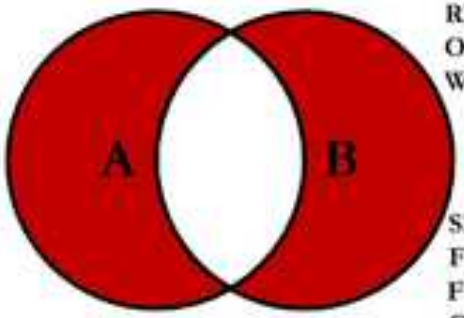

```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```




```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```

© C.L. Moffatt, 2008